



# 编译原理<sup>1</sup>

## Compiler Principles

### Lecture 6

## Semantic Analysis: Intro & SDD & SDT

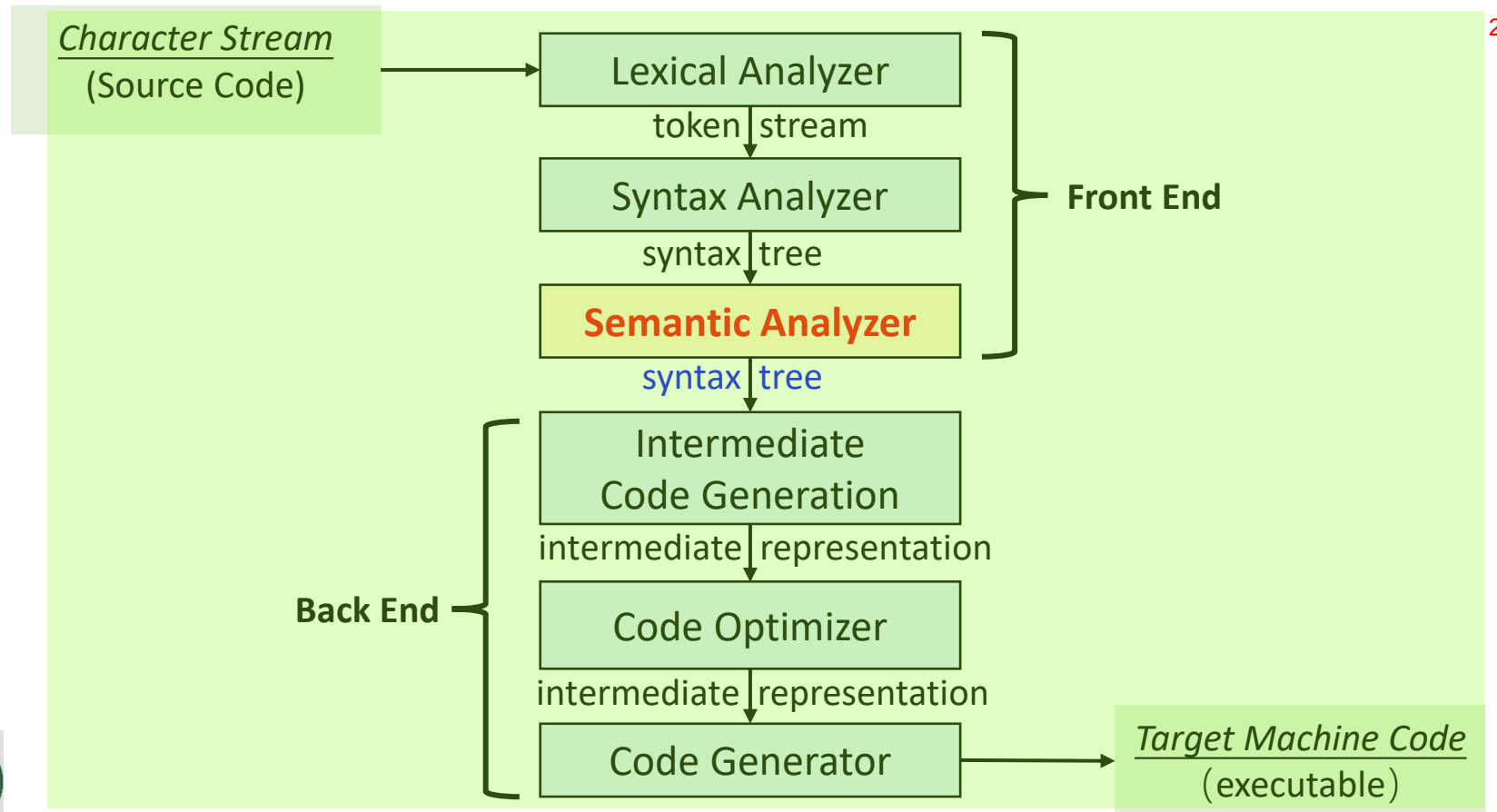
---

赵帅<sup>2</sup>

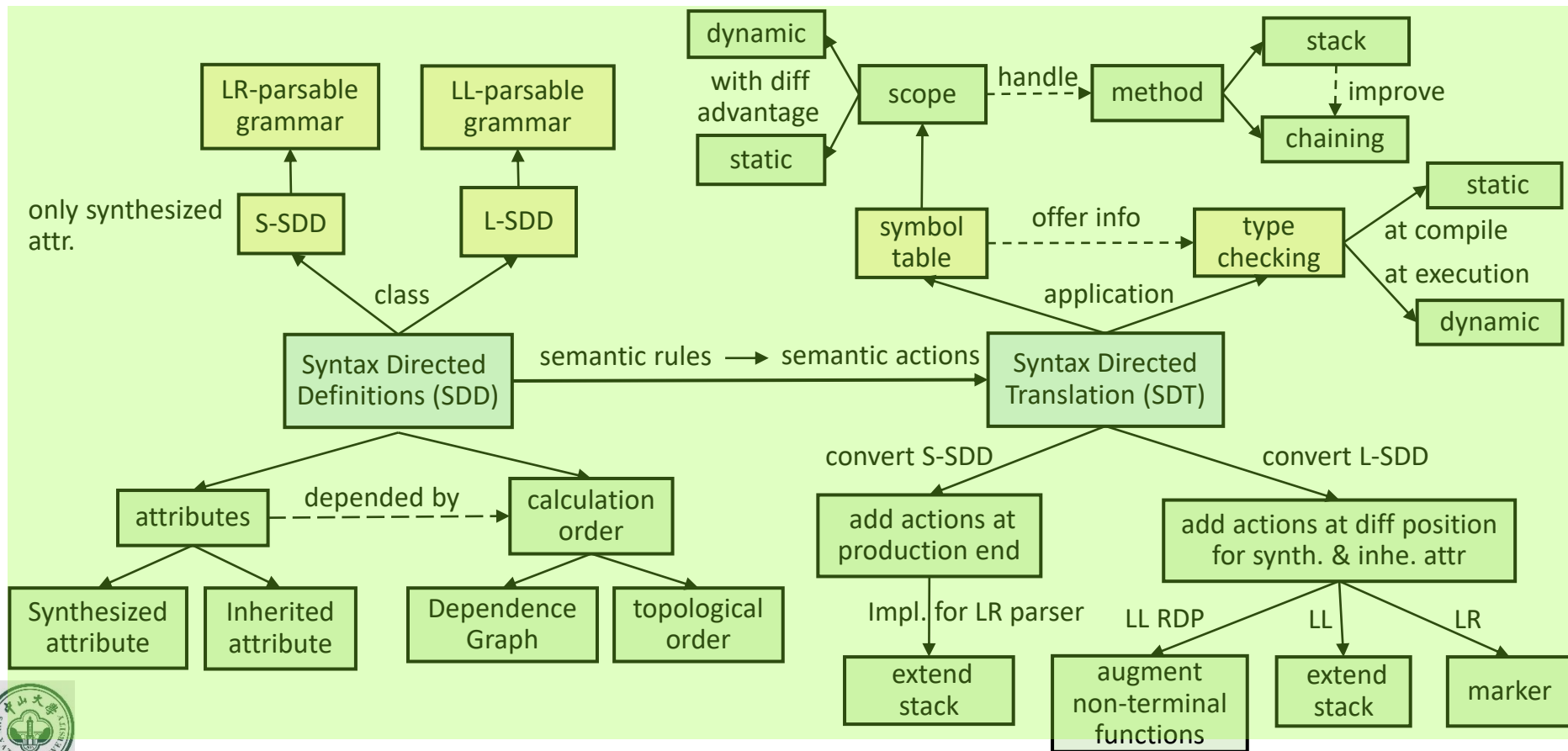
计算机学院<sup>3</sup>

中山大学<sup>4</sup>

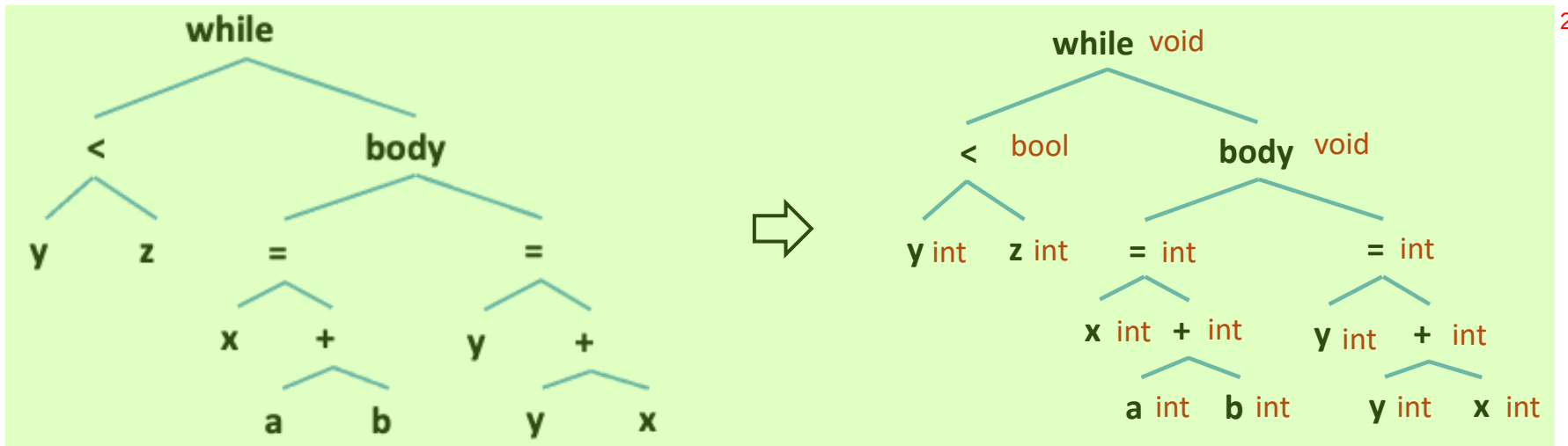
# Compilation Phases [编译阶段]



# Content<sup>1</sup>



# Compilation Phases[编译阶段]<sup>1</sup>



Abstract Syntax Tree(AST)<sup>3</sup>

Annotated AST/Decorated AST<sup>4</sup>  
[带标注的抽象语法树]

# Why Semantic Analysis? [语义分析]<sup>1</sup>



- Because programs use symbols (a.k.a. identifiers)<sup>2</sup>

- ◆ Identifiers require **context** to figure out the meaning<sup>3</sup>

- Consider the English sentence: “He ate it”<sup>4</sup>

- ◆ This sentence is syntactically correct<sup>5</sup>

- ◆ But it makes sense only in the context of a previous sentence: “Sam bought a pizza.” (what if “Sam bought a car.”?)<sup>6</sup>

- Semantic analysis<sup>7</sup>

- ◆ Associates **identifiers** with **objects** they refer to[关联]<sup>8</sup>

- “He” --> “Sam”<sup>9</sup>

- “it” --> “pizza”<sup>10</sup>

- ◆ Checks whether identifiers are used correctly[检查]<sup>11</sup>

- “He” and “it” refer to some object: **def-use check**<sup>12</sup>

- “it” is a type of object that can be eaten: **type check**<sup>13</sup>



# Why Semantic Analysis?<sup>1</sup>

- Semantic of a language is more difficult to describe than syntax<sup>2</sup>  
[语义比语法更难描述]
  - ◆ Syntax: describes the **proper form** of the programs [仅形式]<sup>3</sup>
  - ◆ Semantics: defines **the meaning of programs** (i.e., what each program does when it executes) [到意义]<sup>4</sup>
- Context cannot be analyzed using a CFG parser[CFG不能分析上下文信息]<sup>5</sup>
  - ◆ Associating IDs with objects require expressing the pattern:<sup>6</sup>  
 $\{wcw \mid w \in (a|b)^*\}$ <sup>7</sup>
    - The first **w** represents the definition of an ID<sup>8</sup>
    - The **c** represents arbitrary intervening code<sup>9</sup>
    - The second **w** represents the use of the ID<sup>10</sup>



# Semantic Analysis<sup>1</sup>

- Deeper check into the source program[对程序进一步分析]<sup>2</sup>
  - ◆ Last stage of the front end[前端最后阶段]<sup>3</sup>
  - ◆ Compiler's last chance to reject incorrect programs[最后拒绝机会]<sup>4</sup>
  - ◆ **Verify properties** that aren't caught in earlier phases<sup>5</sup>
    - Variables declaration before use [先声明后使用]<sup>6</sup>
    - Type consistency when using IDs [变量类型一致]<sup>7</sup>
    - Correct expressions types [表达式类型]<sup>8</sup>
- Gather useful info about program for later phases[收集后续信息]<sup>9</sup>
  - ◆ Determine **what variables** are meant by each identifier<sup>10</sup>
  - ◆ Build an **internal representation** of inheritance hierarchies<sup>11</sup>
  - ◆ Count **how many variables** are in scope at each point<sup>12</sup>



# Semantic Analysis: Implementation<sup>1</sup>

- Attribute grammars[属性文法]<sup>2</sup>

- ◆ One-pass compilation<sup>3</sup>

- Semantic analysis is done along with parsing<sup>4</sup>

- ◆ Augment rules to do checking during parsing<sup>5</sup>

- ◆ The approach suggested in the Compilers book<sup>6</sup>

- AST walk[语法树遍历]<sup>7</sup>

- ◆ Two-pass compilation<sup>8</sup>

- First pass digests the syntax and builds a parse tree<sup>9</sup>

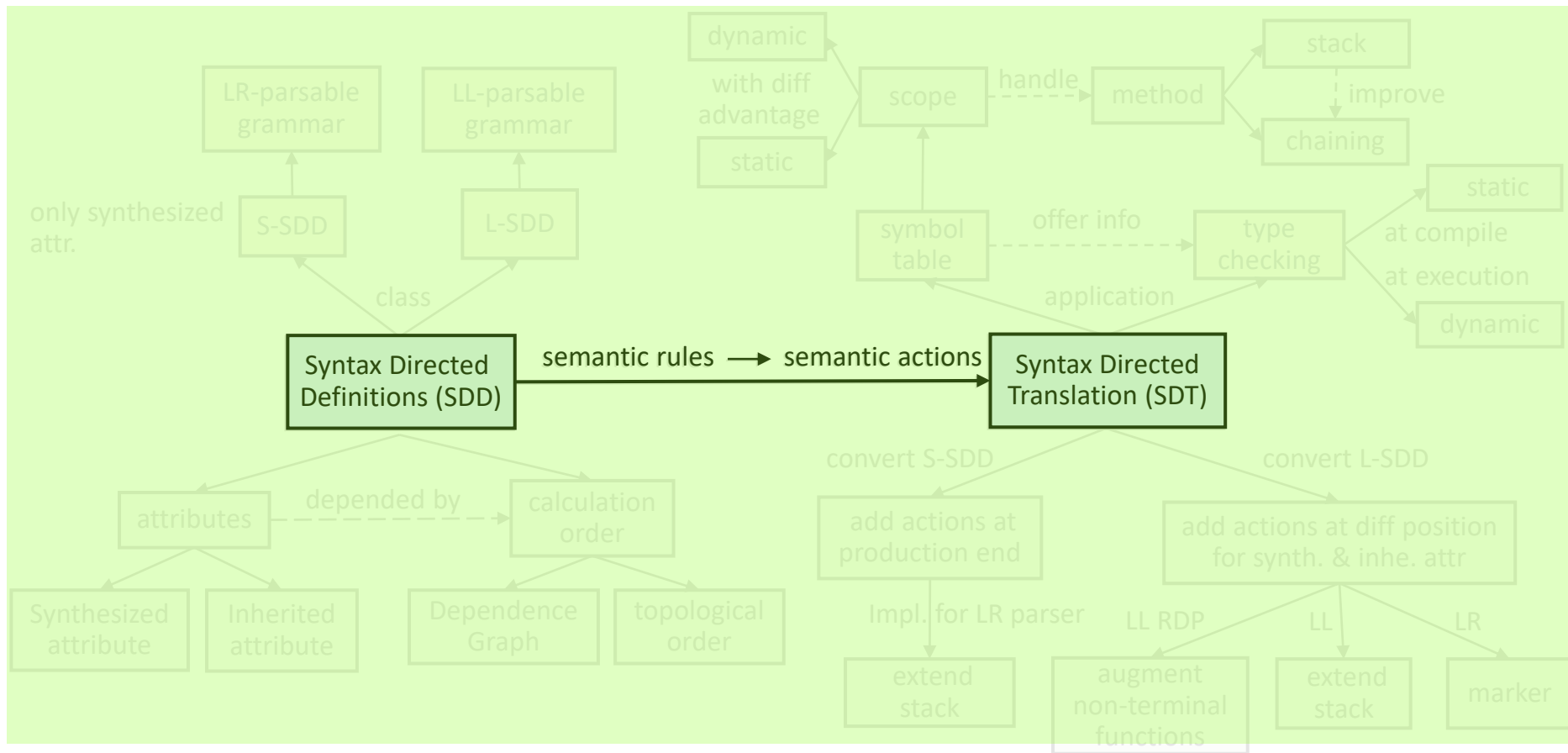
- The second pass traverses the tree to verify that the program respects all semantic rules<sup>10</sup>

- ◆ Strict phase separation of Syntax Analysis and Semantic Analysis<sup>11</sup>



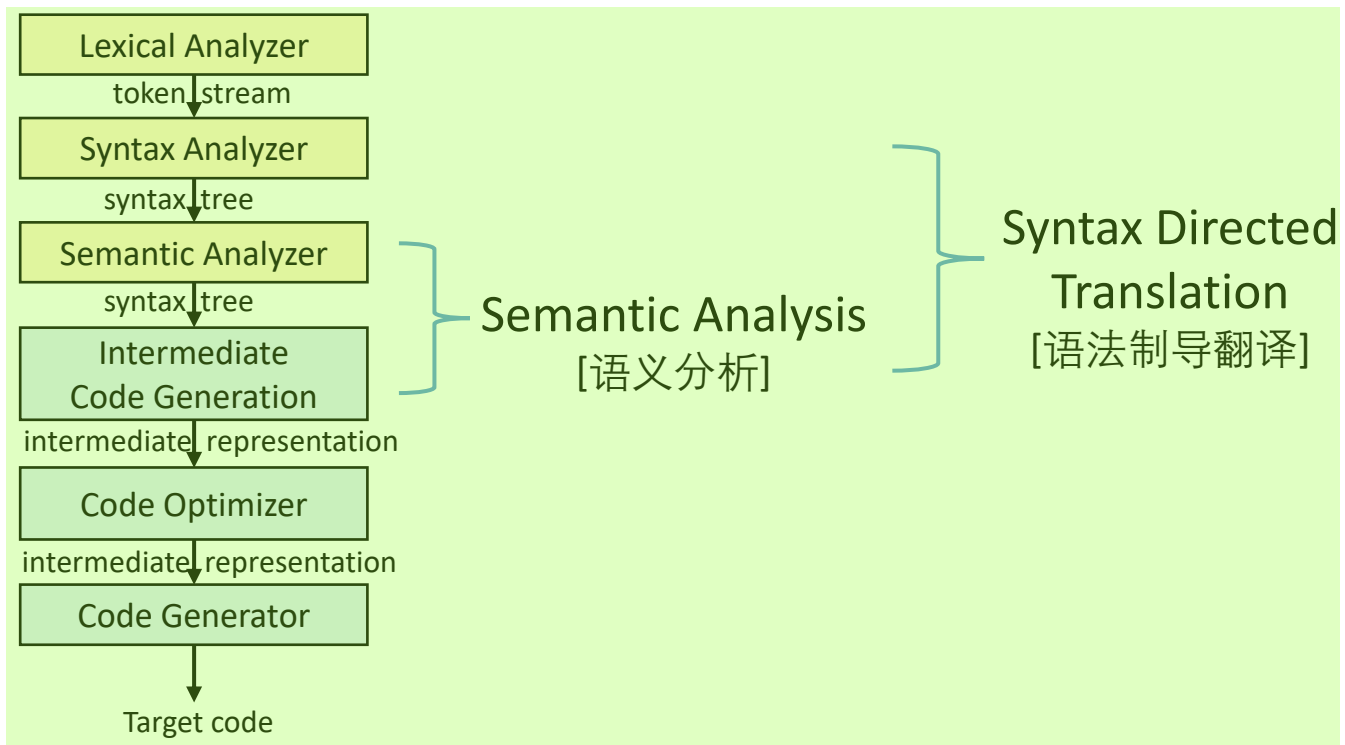


# Content<sup>1</sup>



# Syntax Directed Translation<sup>1</sup>

## • Syntax Directed Translation[语法制导翻译]<sup>2</sup>



# Syntax Directed Translation<sup>1</sup>[语法制导翻译]



- Translate based on the program's grammar structure<sup>2</sup>[语法结构]

- ◆ **Syntactic structure**: structure of a program given by grammar<sup>3</sup>[文法]
- ◆ The parsing process and parse trees are used to direct semantic analysis and the translation of the program, i.e., **CFG-driven translation**<sup>4</sup>[CFG驱动的翻译]

- How? Augment the grammar used in parser<sup>5</sup>[对语法的进一步加强]:

- ◆ Attach **semantic attributes**<sup>6</sup>[语义属性] to each grammar symbol
  - The attributes describe the symbol properties<sup>7</sup>[符号特征]
  - An attribute has a name and an associated value: a string, a number, a type, a memory location, an assigned register...<sup>8</sup>
- ◆ For each grammar production, give **semantic rules or actions**<sup>9</sup>[语义规则或动作]
  - The actions describe how to **compute the attribute values** associated<sup>10</sup> with each symbol in a production



- **Attributes can represent anything depending on the task**<sup>2</sup>[属性可以表示任意含义]

- ◆ If computing expression: a number (value of expression)<sup>3</sup>
- ◆ If building AST: a pointer (pointer to AST for expression)<sup>4</sup>
- ◆ If generating code: a string (assembly code for expression)<sup>5</sup>
- ◆ If type checking: a type (type for expression)<sup>6</sup>

- **Attributes are associated directly with the grammar symbols**<sup>7</sup>

- **Format:  $X.a$**  ( $X$  is a **symbol**,  $a$  is one of its **attributes**)<sup>8</sup>



Associating semantic rules with grammar rules (productions)<sup>2</sup>  
involves two concepts:

- **Syntax Directed Definitions (SDD)** [语法制导定义]<sup>3</sup>

- **Syntax Directed Translation scheme (SDT)** [语法制导翻译方案]<sup>4</sup>



# Syntax Directed Definitions (SDD)<sup>1</sup>

- **Syntax Directed Definitions (SDD)** [语法制导定义]<sup>2</sup>

- ◆ A syntax-directed definition (SDD) is a **context-free grammar**(CFG) together with **attributes and rules**<sup>3</sup>

- ◆ Add **Attributes + semantic rules**[语义规则] in CFG<sup>4</sup>

- ◆ A generalization of CFG<sup>5</sup>

- **Attributes** for grammar symbols [文法符号和语义属性关联]<sup>6</sup>

- **Semantic rules** for productions [产生式和语义规则关联]<sup>7</sup>

| Productions             | Attributes              | Semantic rules            |
|-------------------------|-------------------------|---------------------------|
| $E \rightarrow E1 + E2$ | $E.val; E1.val; E2.val$ | $E.val = E1.val + E2.val$ |
| $E \rightarrow id$      | $E.val; id.lexval$      | $E.val = id.lexval$       |

<sup>8</sup>

# Syntax Directed Translation (SDT)<sup>1</sup>

## • Syntax Directed Translation scheme (SDT) [语法制导翻译方案]<sup>2</sup>

- ◆ SDT is a CFG with program fragments embedded in the right part of the production, and these program fragments are called **semantic actions**.<sup>3</sup>
- ◆ Attributes + semantic actions [语义动作]<sup>4</sup>
- ◆ Example actions for computing the value of an expression<sup>5</sup>

```
E -> E1 + E2 {E.val = E1.val + E2.val}6  
E -> id {E.val = id.lexval}
```

...

The position of a semantic action in a production determines<sup>7</sup>  
the execution point of the action

# SDD v.s. SDT<sup>1</sup>

- SDD[语法制导定义]: 是CFG的推广, 翻译的高层次规则说明<sup>2</sup>
  - ◆ A CFG grammar together with attributes and semantic rules<sup>3</sup>
    - ◆ A subset of them are also called attribute grammars[属性文法]<sup>4</sup>
  - ◆ Semantic rules imply **no order** to attribute evaluation<sup>5</sup>
- SDT[语法制导翻译方案]: SDD的补充, 具体翻译实施方案<sup>6</sup>
  - ◆ An executable specification of the SDD<sup>7</sup>
    - Program fragments are attached to different points in production rules<sup>8</sup>
  - ◆ The **execution order** is important<sup>9</sup>

| Grammar                  | SDD                         | SDT <sup>10</sup>                                      |
|--------------------------|-----------------------------|--------------------------------------------------------|
| D -> TL                  | L.inh = T.type              | D ->T {L.inh = T.type} L                               |
| T -> int                 | T.type = int                | T -> int {T.type = int}                                |
| T -> float               | T.type = float              | T -> float {T.type = float}                            |
| L -> L <sub>1</sub> , id | L <sub>1</sub> .inh = L.inh | L -> {L <sub>1</sub> .inh = L.inh} L <sub>1</sub> , id |



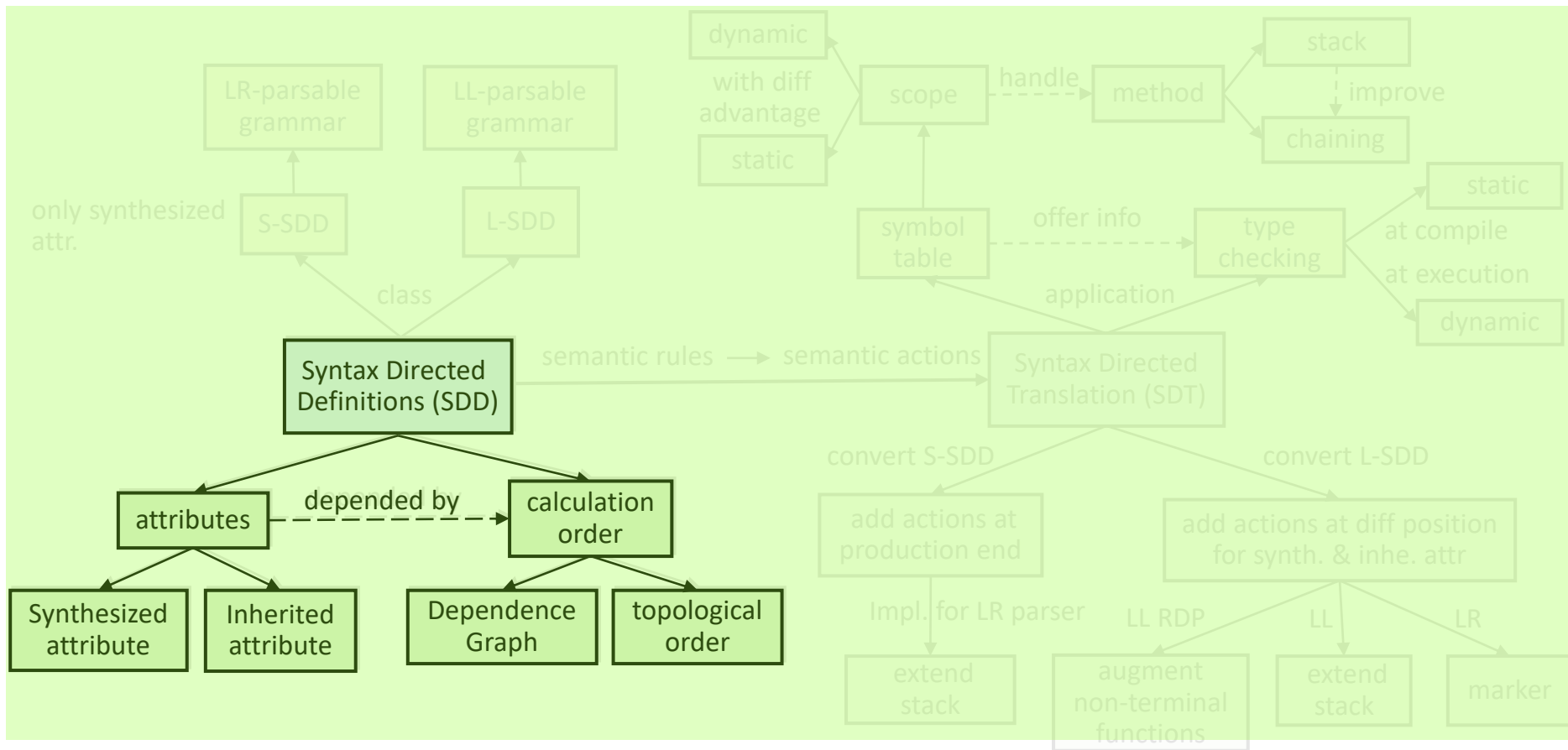
# SDD v.s. SDT<sup>1</sup>

- Syntax:  $A \rightarrow \alpha \{action_1\} \beta \{action_2\} \gamma \dots$ <sup>2</sup>
- Actions are executed “**at that point**” in the RHS<sup>3</sup>
  - ◆  $action_1$  executes after  $\alpha$  has been produced but before  $\beta$ <sup>4</sup>
  - ◆  $action_2$  executes after  $\alpha$ ,  $action_1$ ,  $\beta$  but before  $\gamma$ <sup>5</sup>
- Semantic rule v.s. Action[语义规则 vs. 语义动作]<sup>6</sup>
  - ◆ Semantic rules are not associated with locations in RHS<sup>7</sup>
    - SDD doesn't impose any order other than dependences
  - ◆ Location of action in RHS specifies when it should occur<sup>8</sup>
    - SDT specifies the execution order and time of each action

$A \rightarrow \{ \dots \} X \{ \dots \} Y \{ \dots \}$ <sup>9</sup>

Semantic Actions

# Content<sup>1</sup>



# Syntax Directed Definitions (SDD)<sup>1</sup>

- SDD是CFG的增广<sup>2</sup>

- ◆ Grammar symbols together with semantic attributes<sup>3</sup>
- ◆ Productions associated with a set of semantic rules to compute the value of an expression<sup>4</sup>

- SDD has two types of attributes [文法符号的两种属性]<sup>5</sup>

- ◆ Synthesized attribute [综合属性]<sup>6</sup>
- ◆ Inherited attribute [继承属性]<sup>7</sup>



# Syntax Directed Definitions (SDD)<sup>1</sup>

- **Synthesized attribute**[综合属性]<sup>2</sup>

- ◆ Defined by a semantic rule associated with the production at N<sup>3</sup>

- The production must have A as its head (i.e.,  $A \rightarrow \dots$ )<sup>4</sup>

- ◆ A synthesized attribute of node N is defined only **by attributes of N's children**<sup>5</sup>  
**and N itself**[子节点或自身]

- **Inherited attribute**[继承属性]<sup>6</sup>

- ◆ Defined by a semantic rule associated with the production at the parent of N<sup>7</sup>

- The production must have A as a symbol in its body (i.e.,  $\dots \rightarrow \dots A \dots$ )

- ◆ An inherited attribute of node N is defined only **by attribute values at N's**<sup>8</sup>  
**parent, N itself, and N's siblings**[父节点、自身或兄弟节点]

# Synthesized Attribute[综合属性]<sup>1</sup>

- Synthesized attribute for non-terminal A of parse-tree node N[非终结符的综合属性]<sup>2</sup>

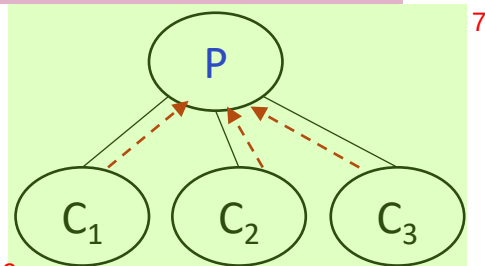
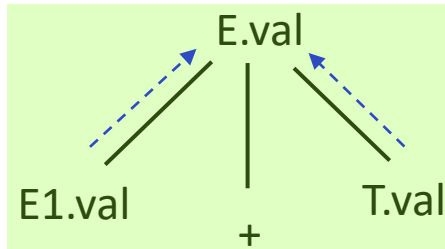
- ◆ Only defined by N's children and N itself<sup>3</sup>

- Passed up the tree<sup>4</sup>

- $P.\text{syn\_attr} = f(P.\text{attrs}, C_1.\text{attrs}, C_2.\text{attrs}, C_3.\text{attrs})$ <sup>5</sup>

- ◆ Example<sup>6</sup>

| Productions            | Semantic rules                                |
|------------------------|-----------------------------------------------|
| $E \rightarrow E1 + T$ | $E.\text{val} = E1.\text{val} + T.\text{val}$ |



- Terminals **CAN** have synthesized attributes[终结符可以具有综合属性]<sup>10</sup>

- ◆ Lexical values supplied by the lexical analysis<sup>12</sup>

- ◆ Thus, no semantic rules in SDD for terminals<sup>13</sup>

# Inherited Attribute[继承属性]<sup>1</sup>

- Inherited attribute for non-terminal A of parse-tree node N[非终结符继承属性]<sup>2</sup>

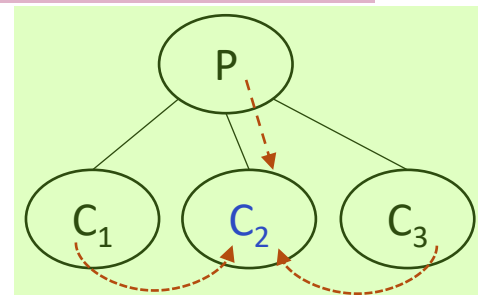
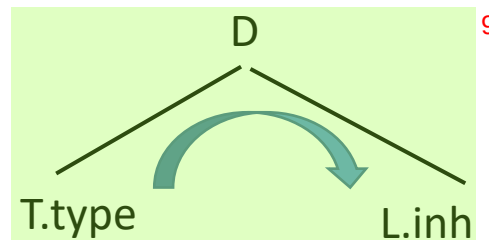
- ◆ Only defined by N's parent, N's siblings and N itself<sup>3</sup>

- Passed down a parse tree<sup>4</sup>

- $C_2.inh\_attr = f(P.attrs, C_1.attrs, C_2.attrs, C_3.attrs)$ <sup>5</sup>

- ◆ Example<sup>7</sup>

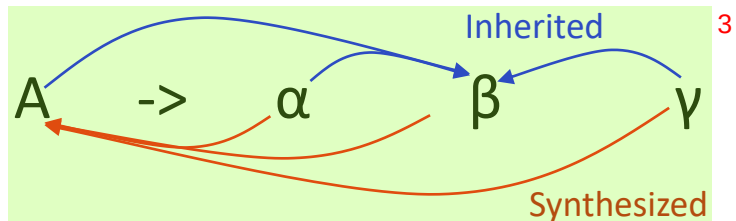
| Productions         | Semantic rules   |
|---------------------|------------------|
| $D \rightarrow T L$ | $L.inh = T.type$ |

<sup>8</sup>

- Terminals **CANNOT** have inherited attributes[终结符无继承属性]<sup>10</sup>

- ◆ Only synthesized attributes from lexical analysis<sup>11</sup>

- Attribute dependencies in a production rule[产生式中的属性依赖]<sup>2</sup>



- SDD has rule of the form for each grammar production<sup>4</sup>

- ◆  $b = f(A.attrs, \alpha.attrs, \beta.attrs, \gamma.attrs)$

- $b$  is either an attribute in LHS (an attribute of  $A$ )<sup>5</sup>

- ◆ In which case  $b$  is a **synthesized** attribute<sup>6</sup>

- ◆ Why? From  $A$ 's perspective  $\alpha, \beta, \gamma$  are children<sup>7</sup>

- Or,  $b$  is an attribute in RHS (e.g., of  $\beta$ )<sup>8</sup>

- ◆ In which case  $b$  is an **inherited** attribute<sup>9</sup>

- ◆ Why? From  $\beta$ 's perspective  $A, \alpha, \gamma$  are parent or siblings<sup>10, 11</sup>



# Example: Synthesized attribute<sup>1</sup>

## Attribute grammar for simple integer arithmetic expression<sup>2</sup>

| Grammar rule                 | Semantic Rules <sup>3</sup> |
|------------------------------|-----------------------------|
| $E \rightarrow E_1 + T$      | $E.val = E_1.val + T.val$   |
| $E \rightarrow T$            | $E.val = T.val$             |
| $T \rightarrow T_1 * F$      | $T.val = T_1.val * F.val$   |
| $T \rightarrow F$            | $T.val = F.val$             |
| $F \rightarrow (E)$          | $F.val = E.val$             |
| $F \rightarrow \text{dight}$ | $F.val = \text{dight.val}$  |

The *val* attribute is synthesized<sup>4</sup>



# Example: Inherited Attribute<sup>1</sup>

## Attribute grammar for variable<sup>2</sup>

| Grammar rule             | Semantic Rules <sup>3</sup>                                 |
|--------------------------|-------------------------------------------------------------|
| decl -> type varlist     | varlist.dtype = type.dtype                                  |
| type -> int              | type.dtype = integer                                        |
| type -> float            | type.dtype = real                                           |
| varlist1 -> id, varlist2 | id.dtype = varlist1.dtype                                   |
| varlist -> id            | varlist2.dtype = varlist1.dtype<br>id.dtype = varlist.dtype |

The *dtype* attribute is inherited<sup>4</sup>

# The Concepts<sup>1</sup>

- Side effect[副作用]<sup>2</sup>

- ◆ 一般属性值计算（基于属性值或常量进行的）之外的功能<sup>3</sup>
- ◆ 例如：code generation, print results, modify symbol table ...<sup>4</sup>

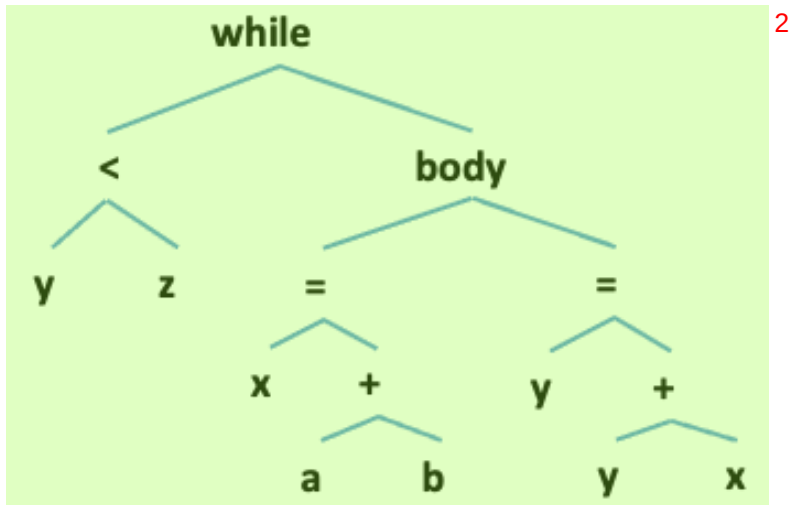
- Attribute grammar[属性文法]<sup>5</sup>

- ◆ 一个没有副作用的SDD<sup>6</sup>
- ◆ The rules define the value of an attribute purely in terms of the value of other attributes and constants[属性文法的规则仅仅通过其他属性值和常量来定义一个属性值]<sup>7</sup>

- Annotated parse-tree[标注分析树]<sup>8</sup>

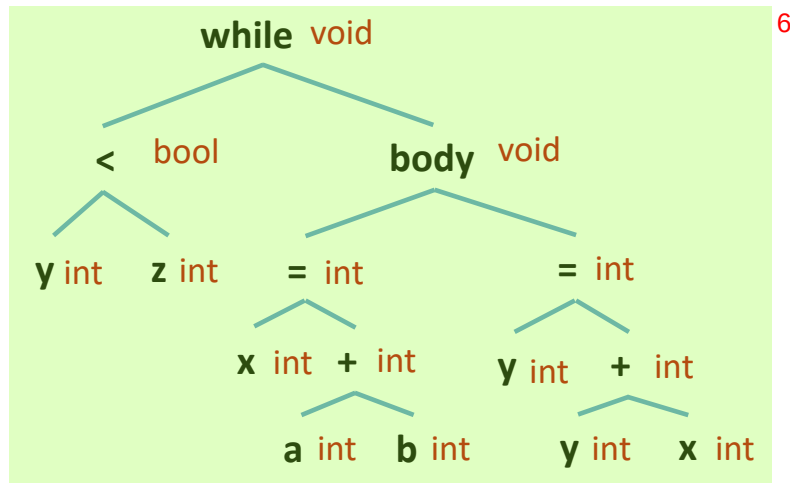
- ◆ 每个节点都带有属性值的分析树<sup>9</sup>
  - A parse tree showing the value(s) of its attribute(s)<sup>10</sup>
- ◆ a.k.a., attribute parse tree[属性分析树]<sup>11</sup>

# Compilation Phases<sup>1</sup>[编译阶段]



Abstract Syntax Tree(AST)<sup>3</sup>

A very simple illustrative case!<sup>5</sup>



Annotated AST/Decorated AST<sup>7</sup>  
[带标注的抽象语法树]

# Semantic Analysis Revisit<sup>1</sup>

- **SDD** (Syntax-directed definition)<sup>2</sup>

- CFG + **attributes** for each symbol + **semantic rules** *for* each production<sup>3</sup>

- **SDT** (Syntax-directed definition)<sup>4</sup>

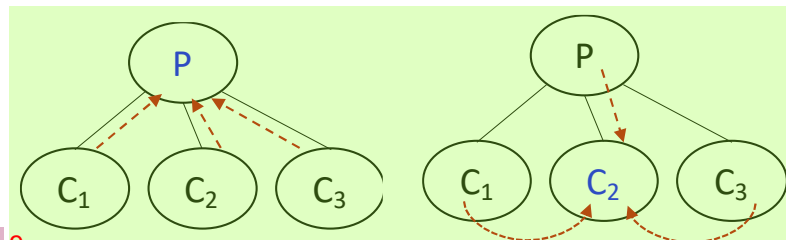
- CFG + **attributes** for each symbol + **{semantic actions}** *in* each production<sup>5</sup>

- **Synthesized attribute**<sup>6</sup>

- Terminals **CAN** have synthesized attributes<sup>7</sup>

- **Inherited Attribute**<sup>8</sup>

- Terminals **CANNOT** have inherited attributes<sup>9</sup>



## Grammar<sup>11</sup>

D -> TL<sup>13</sup>

T -> int<sup>15</sup>

T -> float<sup>16</sup>

L -> L1, id<sup>23</sup>

## SDD (rules)<sup>12</sup>

L.inh = T.type<sup>14</sup>

T.type = int<sup>17</sup>

T.type = float<sup>18</sup>

L1.inh = L.inh<sup>21</sup>

## SDT (actions)<sup>19</sup>

D -> T {L.inh = T.type} L<sup>22</sup>

T -> int {T.type = int}<sup>24</sup>

T -> float {T.type = float}<sup>25</sup>

L -> {L1.inh = L.inh} L1, id<sup>26</sup>



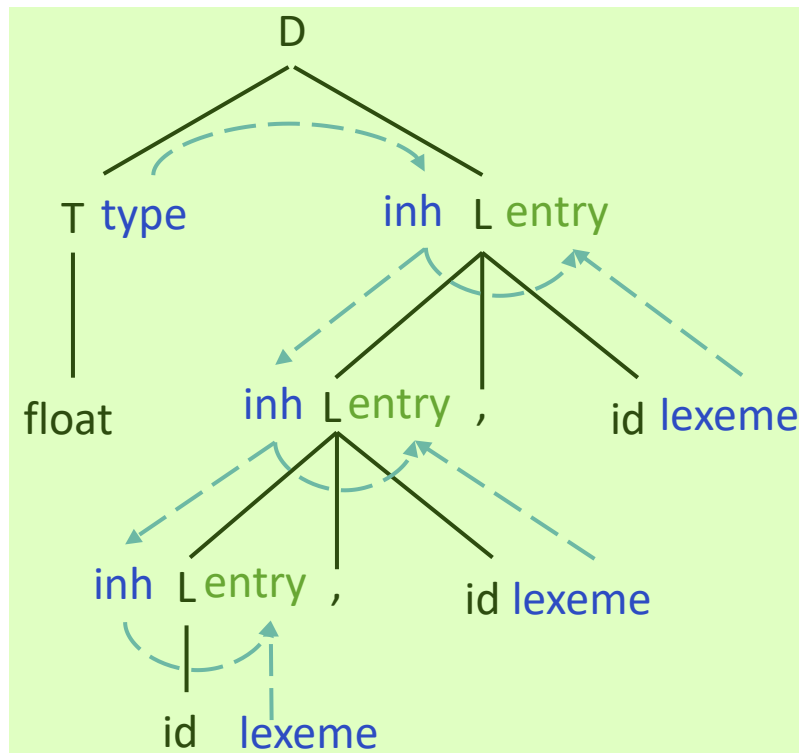
## 29

# Example: Dependency Graph<sup>1</sup>

## SDD:<sup>2</sup>

| Productions                         | Semantic rules                             |
|-------------------------------------|--------------------------------------------|
| D $\rightarrow$ T L                 | L.inh = T.type                             |
| T $\rightarrow$ int                 | T.Type = int                               |
| T $\rightarrow$ float               | T.Type = float                             |
| L $\rightarrow$ L <sub>1</sub> , id | L1.inh = L.inh<br>addtype(id.entry, L.inh) |
| L $\rightarrow$ id                  | addtype(id.entry, L.inh)                   |

Input:<sup>4</sup>  
float a, b, c



'entry' is dummy attribute for the addtype()<sup>6</sup>

# Evaluation Order<sup>1</sup>[属性值计算顺序]

- Ordering the evaluation of attributes<sup>2</sup>[计算顺序]

- ◆ Dependency graph characterizes possible orders in which we can evaluate the attributes at the various nodes of a parse-tree<sup>3</sup>[依赖图描述了属性计算顺序]

- If the graph has an edge from node **M** to node **N**, then the attribute associated with **M** must be evaluated before **N**<sup>4</sup>[用图的边来确定计算顺序]

- ◆ Thus, the only allowable orders of evaluation are those sequences of nodes  $N_1, N_2, \dots, N_k$  such that if there is an edge from  $N_i$  to  $N_j$  and  $i < j$ <sup>5</sup>

- ◆ This order embeds a directed graph into a linear order, called a **topological sort**<sup>6</sup>[拓扑排序] of the graph

- If there's any cycle in the graph, then there are no topological sorts, i.e., no way to evaluate the SDD on this parse tree<sup>7</sup>

- If there are no cycles, then there is always at least one topological sort<sup>8</sup>



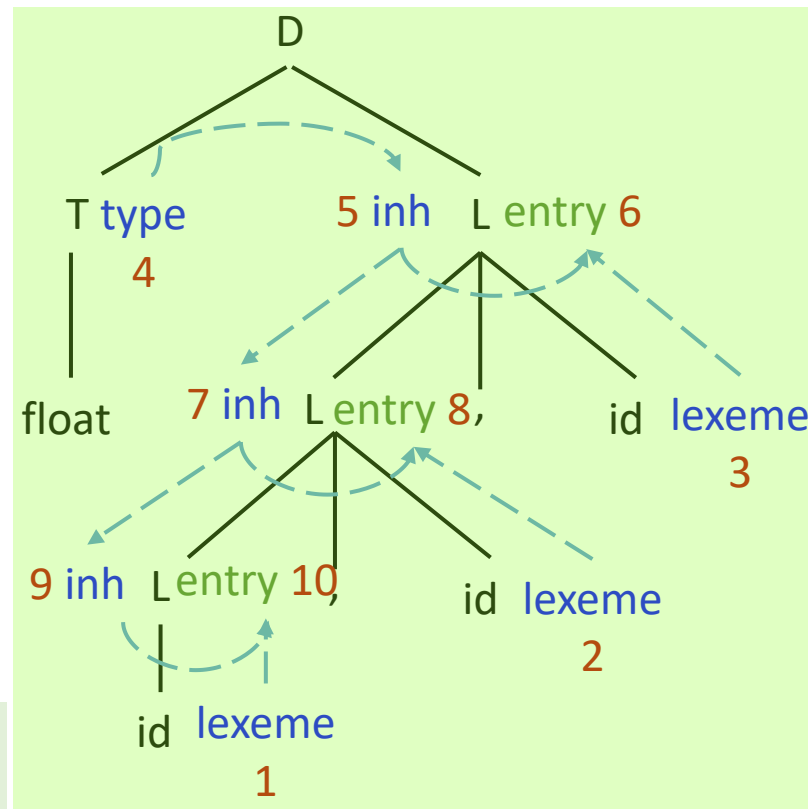
# Example: Evaluation Order<sup>1</sup>

## SDD:<sup>2</sup>

| Productions                         | Semantic rules                             |
|-------------------------------------|--------------------------------------------|
| D $\rightarrow$ T L                 | L.inh = T.type                             |
| T $\rightarrow$ int                 | T.type = int                               |
| T $\rightarrow$ float               | T.type = float                             |
| L $\rightarrow$ L <sub>1</sub> , id | L1.inh = L.inh<br>addtype(id.entry, L.inh) |
| L $\rightarrow$ id                  | addtype(id.entry, L.inh)                   |

Input:<sup>4</sup>  
float a,b,c

Topological sort:<sup>5</sup>  
1, 2, 3, 4, 5, 6, 7, 8, 9, 10





# Evaluation Order (cont.)<sup>1</sup>

- Before evaluating an attribute at a node of a parse tree, we must evaluate **all attributes it depends on** [依赖关系]<sup>2</sup>
  - ◆ **Synthesized**: evaluate **children first**, then the node itself<sup>3</sup>
  - ◆ **Inherited**: evaluate **parent and dependent siblings** first, then the node itself<sup>4</sup>
  - ◆ For SDD's with both inherited and synthesized attributes, there's no guarantee that there is even one evaluation order<sup>5</sup>
- Difficult to check circularities in a dependency graph<sup>6</sup>
  - ◆ But, there are **subclasses of SDD's** that guarantee an evaluation order<sup>7</sup>
    - Such classes do not permit graphs with cycles<sup>8</sup>

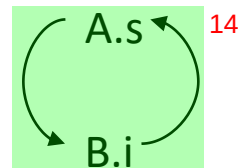
Production<sup>9</sup>

$A \rightarrow B$ <sup>11</sup>

Semantic Rules<sup>12</sup>

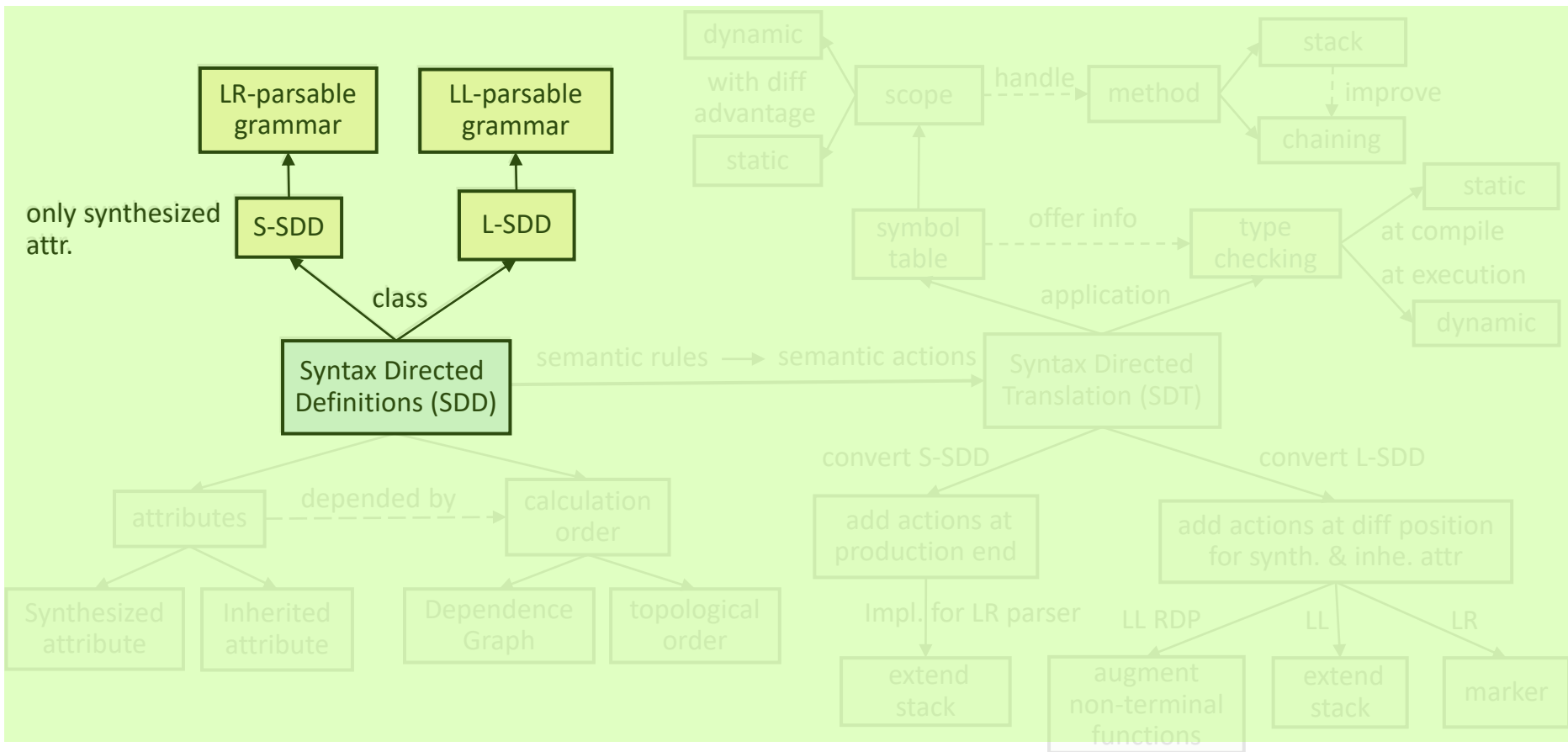
$A.s = B.i$ ;<sup>13</sup>

$B.i = A.s + 1$ ;<sup>15</sup>






 中山大學  
 SUN YAT-SEN UNIVERSITY



# S-Attributed Definitions<sup>1</sup>[S-属性定义]

- An SDD is **S-attributed** if every attribute is synthesized<sup>2</sup>[只具有综合属性]

- If an SDD is S-attributed (S-SDD)<sup>3</sup>

- ◆ Can evaluate its attributes in **any bottom-up order** of the nodes of the parse-tree<sup>4</sup>[自底向上的顺序计算属性值]

- ◆ Can be implemented during bottom-up parsing<sup>5</sup>[与自底向上的语法分析同时进行]

| Productions                      | Semantic rules                                 |
|----------------------------------|------------------------------------------------|
| (1) $L \rightarrow E$            | $\text{print}(E.\text{val})$                   |
| (2) $E \rightarrow E_1 + T$      | $E.\text{val} = E_1.\text{val} + T.\text{val}$ |
| (3) $E \rightarrow T$            | $E.\text{val} = T.\text{val}$                  |
| (4) $T \rightarrow T_1 * F$      | $T.\text{val} = T_1.\text{val} * F.\text{val}$ |
| (5) $T \rightarrow F$            | $T.\text{val} = F.\text{val}$                  |
| (6) $F \rightarrow (E)$          | $F.\text{val} = E.\text{val}$                  |
| (7) $F \rightarrow \text{digit}$ | $F.\text{val} = \text{digit}.\text{lexval}$    |

# L-Attributed Definitions [L-属性定义]<sup>1</sup>

• An SDD is **L-attributed** (L-SDD) if<sup>2</sup>

◆ Between the attributes associated with a production body, **dependency graph edges can only go from left to right** [依赖图的边只能从左到右]<sup>3</sup>

◆ Each attribute can be either **synthesized, or inherited** with the rules that suppose  $A \rightarrow X_1X_2...X_{i-1}X_i$ , the inherited attribute  $X_i.a$  only depends on:<sup>4</sup>

1. **Inherited** attributes associated with  $A$  [A的**继承**属性, Why?]<sup>5</sup>

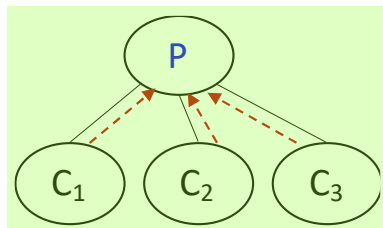
2. Either **syn or inh** attributes of  $X_1, X_2, ..., X_{i-1}$  located to the **left** of  $X_i$  [ $X_i$ 的之前符号的综合/继承属性]<sup>6</sup>

3. Either **syn or inh** attributes of  $X_i$  itself, but **no cycles** formed by the attributes of this  $X_i$  [ $X_i$ 的综合/继承属性 (不能出现环形依赖)]<sup>7</sup>

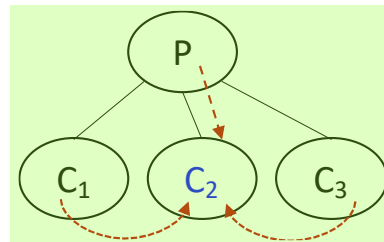
◆ L-SDD是对S-SDD的拓展, Can be implemented during **top-down/bottom-up parsing** [L属性的语义分析可在LL/LR分析中完成]<sup>8</sup>



# L-Attributed Definitions (cont.)<sup>1</sup>



综合属性<sup>3</sup>



继承属性<sup>5</sup>

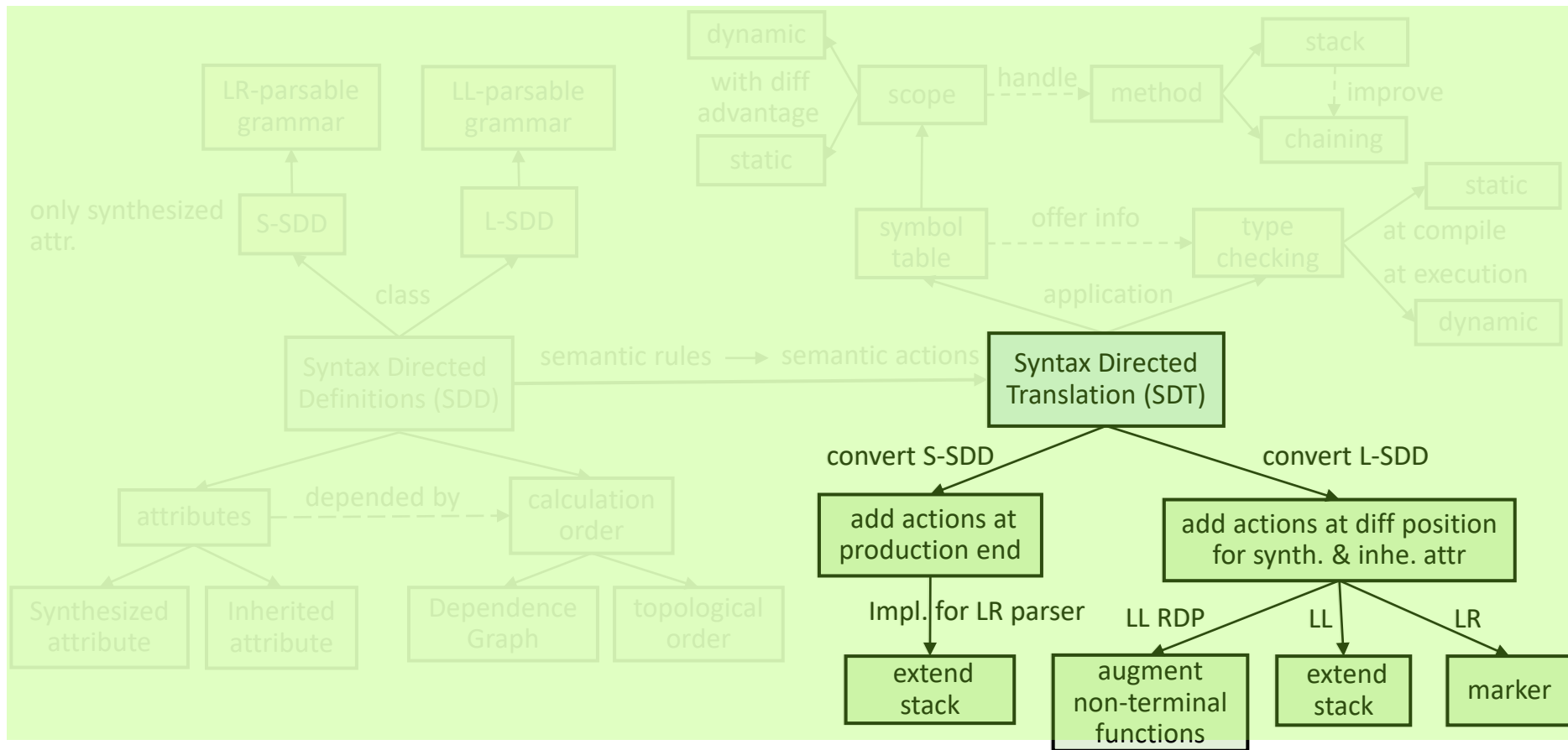
S-SDD or L-SDD?<sup>6</sup>

| Productions        | Semantic rules <sup>7</sup>        |
|--------------------|------------------------------------|
| $A \rightarrow BC$ | $A.s = B.b$<br>$B.i = f(C.c, A.s)$ |

Not S-SDD: B.i is inh<sup>8</sup>  
[S属性文法中不会出现继承属性]

Not L-SDD: A.s is syn arrt, C is right to B<sup>9</sup>  
[L属性文法中, 继承属性 (B.i) 不会依赖综合属性 (A.s)]  
[L属性文法中, RHS 不会出现从右到左的依赖关系]

# Content<sup>1</sup>



• SDT (executable SDD) can be implemented in two ways<sup>2</sup>

◆ **Using a parse tree or AST**[基于预先构建的分析树]<sup>3</sup>

▫ First build a parse tree, and then apply rules or actions at each node while traversing the tree<sup>4</sup>

▫ All SDDs (without cycles) and SDTs can be implemented since the tree can be traversed freely, implements any ordering<sup>5</sup>

◆ **During parsing**, without building a parse tree[语法分析过程中]<sup>6</sup>

▫ Apply rules or actions at each production while parsing<sup>7</sup>

▫ Only a subset of SDDs and SDTs can be implemented<sup>8</sup>

▫ Evaluation ordering restricted to parser derivation order<sup>9</sup>



# Implementation<sup>1</sup>[实现]

- Two important classes of SDD's<sup>2</sup>[两个关键子类]

- ◆ SDD is S-attributed, the underlying grammar is LR-parsable<sup>3</sup>

- ◆ SDD is L-attributed, the underlying grammar is LL-parsable<sup>4</sup>

- For both classes, semantic rules in an SDD can be converted into an SDT with actions that are executed at the right time<sup>5</sup>[允许SDD到SDT的转换]

- ◆ During parsing, an action in a production body is executed as soon as all the grammar symbols to the left of the action have been matched<sup>6</sup>





# Implement S-SDD<sup>1</sup>

## • Convert S-attributed SDD to SDT[SDD到SDT的转换]<sup>2</sup>

- ◆ Place each action at the end of the production[将每个语义动作都放在产生式的最后]<sup>3</sup>
- ◆ SDTs with all actions at the right ends of the production bodies are called **Postfix SDT** [后綴/尾部SDT]<sup>4</sup>

S-SDD<sup>5</sup>

| Productions                 | Semantic rules                 |
|-----------------------------|--------------------------------|
| (1) $L \rightarrow E$       | $print(E.val)$                 |
| (2) $E \rightarrow E_1 + T$ | $E.val = E_1.val + T.val$      |
| (3) $E \rightarrow T$       | $E.val = T.val$                |
| (4) $T \rightarrow T_1 * F$ | $T.val = T_1.val \times F.val$ |
| (5) $T \rightarrow F$       | $T.val = F.val$                |
| (6) $F \rightarrow (E)$     | $F.val = E.val$                |
| (7) $F \rightarrow digit$   | $F.val = digit.lexval$         |

SDT<sup>7</sup>

| CFG with actions                                               |
|----------------------------------------------------------------|
| (1) $L \rightarrow E \{ print(E.val) \}$                       |
| (2) $E \rightarrow E_1 + T \{ E.val = E_1.val + T.val \}$      |
| (3) $E \rightarrow T \{ E.val = T.val \}$                      |
| (4) $T \rightarrow T_1 * F \{ T.val = T_1.val \times F.val \}$ |
| (5) $T \rightarrow F \{ T.val = F.val \}$                      |
| (6) $F \rightarrow (E) \{ F.val = E.val \}$                    |
| (7) $F \rightarrow digit \{ F.val = digit.lexval \}$           |

# Implement S-SDD (cont.)<sup>1</sup>

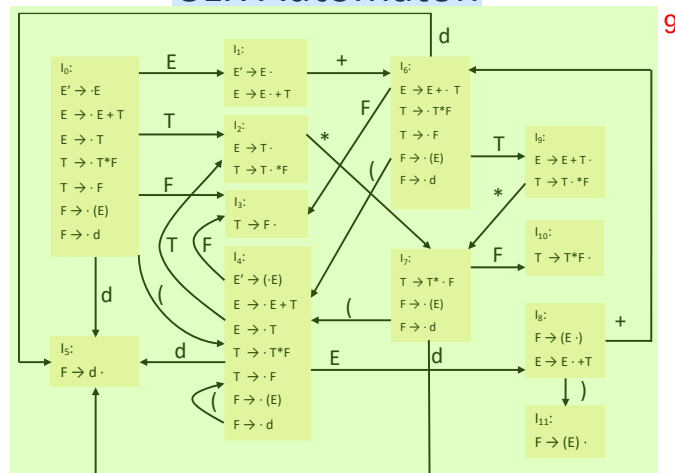
- If the underlying grammar of S-SDD is **LR parsable**<sup>2</sup>
  - ◆ Then the SDT can be implemented during LR parsing<sup>3</sup>
- Implement the converted SDT **by reduction** [借助归约实现]<sup>4</sup>
  - ◆ Executing the action along with the reduction of LHS  $\leftarrow$  RHS<sup>5</sup>

## SDT<sup>6</sup>

### CFG with actions

- (1)  $L \rightarrow E$  {*print*(*E.val*)}
- (2)  $E \rightarrow E_1 + T$  {*E.val* = *E<sub>1</sub>.val* + *T.val*}
- (3)  $E \rightarrow T$  {*E.val* = *T.val*}
- (4)  $T \rightarrow T_1 * F$  {*T.val* = *T<sub>1</sub>.val* \* *F.val*}
- (5)  $T \rightarrow F$  {*T.val* = *F.val*}
- (6)  $F \rightarrow (E)$  {*F.val* = *E.val*}
- (7)  $F \rightarrow \text{digit}$  {*F.val* = *digit.lexval*}

## SLR Automaton<sup>8</sup>

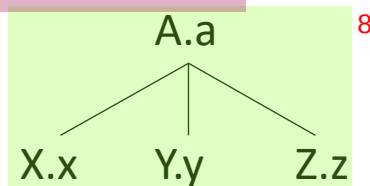


# Extend LR Parse Stack[扩展分析栈]<sup>1</sup>

- Save **synthesized attributes** into the **stack**[栈中额外存放综合属性值]<sup>2</sup>
  - ◆ Place the attributes along with the grammar symbols (or LR states that associated with these symbols) in records on stack<sup>3</sup>
  - ◆ If there are multiple attributes<sup>4</sup>
    - Make the records large enough or by putting pointers to records on the stack [栈记录足够大, 或栈记录中存放指针]<sup>5</sup>

## • Example: $A \rightarrow XYZ$ <sup>6</sup>

- ◆  $x, y, z$  are attributes of  $X, Y, Z$  respectively<sup>7</sup>
- ◆ After the action,  $A$  and its attributes are at the top (i.e.,  $m-2$ )<sup>9</sup>



|            |       |     |           |           |       |
|------------|-------|-----|-----------|-----------|-------|
| State:     | $S_0$ | ... | $S_{m-2}$ | $S_{m-1}$ | $S_m$ |
| Symbol:    | $\$$  | ... | $X$       | $Y$       | $Z$   |
| Attribute: | -     | ... | $X.x$     | $Y.y$     | $Z.z$ |

top



|              |       |     |           |
|--------------|-------|-----|-----------|
| state ->     | $S_0$ | ... | $S_{m-2}$ |
| Symbol ->    | $\$$  | ... | $A$       |
| Attribute -> | -     | ... | $A.a$     |

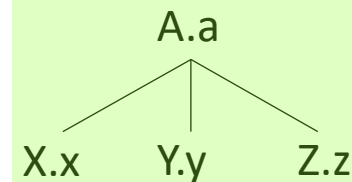
top

# Stack Manipulation<sup>1</sup>[栈操作]

- Rewrite the actions to manipulate the parser stack<sup>2</sup>
  - ◆ The manipulation can be done automatically by the parser<sup>3</sup>

```
stack[top-2].symbol= A4  
stack[top-2].val = f(stack[top-2].val, stack[top-1].val, stack[top].val)top = top-2
```

```
A -> XYZ {A.a= f(X.x,Y.y,Z.z)}5
```



|            |                |     |                  |                  |                |              |
|------------|----------------|-----|------------------|------------------|----------------|--------------|
| State:     | S <sub>0</sub> | ... | S <sub>m-2</sub> | S <sub>m-1</sub> | S <sub>m</sub> | <sup>7</sup> |
| Symbol:    | \$             | ... | X                | Y                | Z              |              |
| Attribute: | -              | ... | X.x              | Y.y              | Z.z            |              |
|            |                |     |                  |                  | top            |              |



|              |                |     |                  |              |
|--------------|----------------|-----|------------------|--------------|
| state ->     | S <sub>0</sub> | ... | S <sub>m-2</sub> | <sup>8</sup> |
| Symbol ->    | \$             | ... | A                |              |
| Attribute -> | -              | ... | A.a              |              |
|              |                |     | top              |              |

# Example<sup>1</sup>

## • Rewrite the actions to manipulate the parser stack<sup>2</sup>

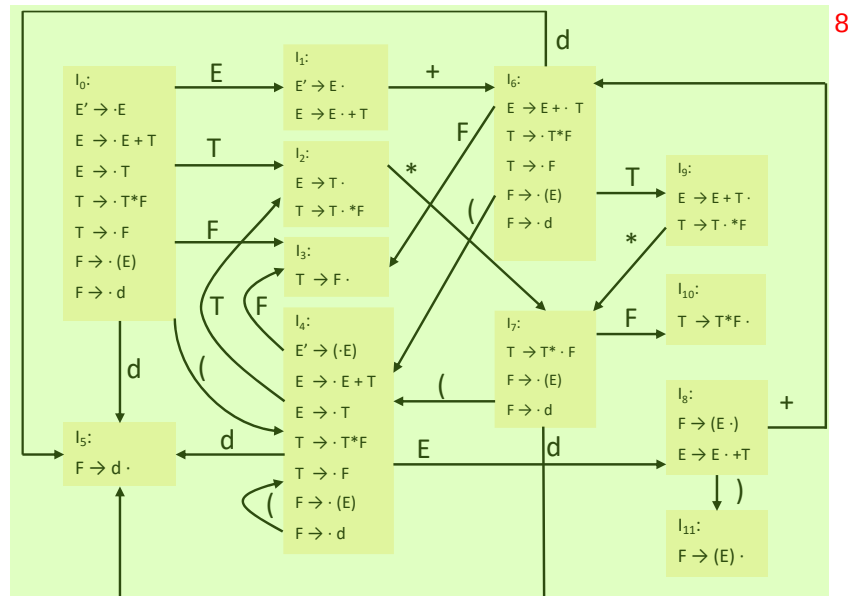
◆ The manipulation can be done automatically by the parser<sup>3</sup>

| Productions                      | Semantic rules                                 | Semantic Actions                                                                                                                                           |
|----------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (1) $L \rightarrow E$            | $\text{print}(E.\text{val})$                   | $\{\text{print}(\text{stack}[\text{top}].\text{val});\}$                                                                                                   |
| (2) $E \rightarrow E_1 + T$      | $E.\text{val} = E_1.\text{val} + T.\text{val}$ | $\{\text{stack}[\text{top}-2].\text{val} = \text{stack}[\text{top}-2].\text{val} + \text{stack}[\text{top}].\text{val};$<br>$\text{top} = \text{top}-2;\}$ |
| (3) $E \rightarrow T$            | $E.\text{val} = T.\text{val}$                  |                                                                                                                                                            |
| (4) $T \rightarrow T_1 * F$      | $T.\text{val} = T_1.\text{val} * F.\text{val}$ | $\{\text{stack}[\text{top}-2].\text{val} = \text{stack}[\text{top}-2].\text{val} * \text{stack}[\text{top}].\text{val};$<br>$\text{top} = \text{top}-2;\}$ |
| (5) $T \rightarrow F$            | $T.\text{val} = F.\text{val}$                  |                                                                                                                                                            |
| (6) $F \rightarrow (E)$          | $F.\text{val} = E.\text{val}$                  | $\{\text{stack}[\text{top}-2].\text{val} = (\text{stack}[\text{top}-1].\text{val});$<br>$\text{top} = \text{top}-2;\}$                                     |
| (7) $F \rightarrow \text{digit}$ | $F.\text{val} = \text{digit}.\text{lexval}$    |                                                                                                                                                            |

# Example<sup>1</sup>

| Productions                      | Semantic Actions                                                          |
|----------------------------------|---------------------------------------------------------------------------|
| (1) $L \rightarrow E$            | { print(stack[top].val); }                                                |
| (2) $E \rightarrow E_1 + T$      | { stack[top-2].val = stack[top-2].val<br>+ stack[top].val; top = top-2; } |
| (3) $E \rightarrow T$            |                                                                           |
| (4) $T \rightarrow T_1 * F$      | { stack[top-2].val = stack[top-2].val<br>* stack[top].val; top = top-2; } |
| (5) $T \rightarrow F$            |                                                                           |
| (6) $F \rightarrow (E)$          | { stack[top-2].val = stack[top-1].val;<br>top = top-2; }                  |
| (7) $F \rightarrow \text{digit}$ |                                                                           |

Input:  $3 * 5 + 4$ <sup>3</sup>



state  $\rightarrow S_0 S_2 S_7 S_{10}$ <sup>4</sup>

Symbol  $\rightarrow \$ T * F$ <sup>5</sup>

Attribute  $\rightarrow - 3 - 5 (\text{top})$ <sup>7</sup>



state  $\rightarrow S_0 S_n$ <sup>9</sup>

Symbol  $\rightarrow \$ T$

Attribute  $\rightarrow - 15 (\text{top})$

# Example<sup>1</sup>

Input:  $3 * 5 + 4$ <sup>2</sup>

## Productions<sup>3</sup>

- (1)  $L \rightarrow E n$
- (2)  $E \rightarrow E1 + T$
- (3)  $E \rightarrow T$
- (4)  $T \rightarrow T_1 * F$
- (5)  $T \rightarrow F$
- (6)  $F \rightarrow (E)$
- (7)  $F \rightarrow \text{digit}$

| Step | States<br>(Illustrative) | Attributes     | Input                 | Code      | Output                       |
|------|--------------------------|----------------|-----------------------|-----------|------------------------------|
| 1    | \$                       | \$             | <b>3 * 5 + 4 n \$</b> |           |                              |
| 2    | \$ 3                     | \$ 3           | <b>* 5 + 4 n \$</b>   | --        | $F \rightarrow \text{digit}$ |
| 3    | \$ F                     | \$ 3           | <b>* 5 + 4 n \$</b>   | --        | $T \rightarrow F$            |
| 4    | \$ T                     | \$ 3           | <b>* 5 + 4 n \$</b>   |           |                              |
| 5    | \$ T *                   | \$ 3 *         | <b>5 + 4 n \$</b>     |           |                              |
| 6    | \$ T * 5                 | \$ 3 * 5       | <b>+ 4 n \$</b>       | --        | $F \rightarrow \text{digit}$ |
| 7    | \$ T * F                 | \$ 3 * 5       | <b>+ 4 n \$</b>       | 3 * 5     | $T \rightarrow T * F$        |
| 8    | \$ T                     | \$ 15          | <b>+ 4 n \$</b>       | --        | $E \rightarrow T$            |
| 9    | \$ E                     | \$ 15          | <b>+ 4 n \$</b>       |           |                              |
| 10   | \$ E +                   | \$ 15 +        | <b>4 n \$</b>         |           |                              |
| 11   | \$ E + 4                 | \$ 15 + 4      | <b>n \$</b>           | --        | $F \rightarrow \text{digit}$ |
| 12   | \$ E + F                 | \$ 15 + 4      | <b>n \$</b>           | --        | $T \rightarrow F$            |
| 13   | \$ E + T                 | \$ 15 + 4      | <b>n \$</b>           | 15 + 4    | $E \rightarrow E + T$        |
| 14   | \$ E                     | \$ 19          | <b>n \$</b>           |           |                              |
| 15   | \$ E <b>n</b>            | \$ 19 <b>n</b> | <b>\$</b>             | print(19) | $L \rightarrow E n$          |
| 16   | \$ L                     | \$ 19          | <b>\$</b>             | accept    |                              |

<sup>5</sup>

# Implement L-SDD<sup>1</sup>

- We have examined **S-SDD** -> **SDT** -> **implementation**<sup>2</sup>

- ◆ S-SDD can be converted to SDT with **actions at production ends**<sup>3</sup>

- ◆ The SDT can be parsed and translated by **bottom-up**, as long as the underlying grammar is **LR-parsable**<sup>4</sup>

- What about the more-general **L-attributed SDD**?<sup>5</sup>

- ◆ Rule for turning L-SDD into an SDT<sup>6</sup>

1. Embed the action that computes the **inherited attributes** for a nonterminal **A**<sup>7</sup> immediately **before the occurrence of A** in the production body
2. Place the actions that compute a **synthesized attribute** for the LHS **at the end of the production body**<sup>8</sup>

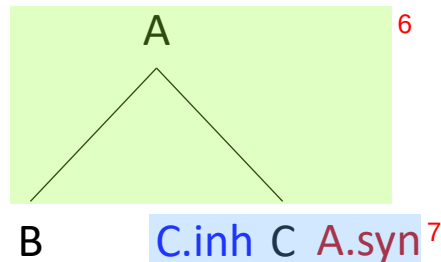


# Example<sup>1</sup>

• **A** -> **B** {C.inh} **C** {A.syn}<sup>2</sup>

◆ C的继承属性: 在C之前<sup>3</sup>

◆ A的综合属性: 在RHS末尾<sup>4</sup>



| Productions                   | Semantic rules                                     |
|-------------------------------|----------------------------------------------------|
| (1) $T \rightarrow FT'$       | $T'.inh = F.val$<br>$T.val = T'.syn$               |
| (2) $T' \rightarrow *FT_1'$   | $T_1'.inh = T'.int * F.val$<br>$T'.syn = T_1'.syn$ |
| (3) $T' \rightarrow \epsilon$ | $T'.syn = T'.inh$                                  |
| (4) $F \rightarrow digit$     | $F.val = digit.lexval$                             |

## SDT

- $T \rightarrow F \{T'.inh = F.val\} T' \{T.val = T'.syn\}$
- $T' \rightarrow *F \{T_1'.inh = T'.int * F.val\} T_1' \{T'.syn = T_1'.syn\}$
- $T' \rightarrow \epsilon \{T'.syn = T'.inh\}$
- $F \rightarrow digit \{F.val = digit.lexval\}$

# Implement L-SDD<sup>1</sup>

- If the underlying grammar is **LL-parsable**, then the **SDT of L-SDD** can be implemented during **LL or LR parsing** [若文法是LL可解析的，则可在LL或LR语法分析过程中实现]<sup>2</sup>

- Semantic translation during **parsing**<sup>3</sup>

- ◆ An **LL** recursive-descent parser[递归下降的LL分析]<sup>4</sup>

- Augment non-terminal functions to both **parsing** and **attributes handling**<sup>5</sup>

- ◆ An **LL** predictive parser[非递归的LL预测分析]<sup>6</sup>

- Extend the **parse stack** to **hold actions** and certain **data items** needed for attribute evaluation<sup>7</sup>

- ◆ An **LR** parser[LR分析]<sup>8</sup>

- Rewrite grammars to involve **marker**<sup>9</sup>

- For the top-down ones, we need to **eliminate left recursion first!**<sup>10</sup>



# Eliminating left recursion<sup>1</sup>

- A motivating example: a simple case<sup>2</sup>

• Trick: **treating actions as terminals** if **they do not calculate** any attributes.<sup>3</sup>

$$E^* \Rightarrow E + T + T + T \dots$$
<sup>4</sup>

$$A \rightarrow A\alpha \mid \beta \Leftrightarrow A \rightarrow \beta A'; A' \rightarrow \alpha A' \mid \varepsilon$$
<sup>5</sup>

$$\begin{aligned} E &\rightarrow E + T \\ E &\rightarrow T \end{aligned}$$

$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow + T R \mid \varepsilon \end{aligned}$$
<sup>6</sup>

$$\begin{aligned} E &\rightarrow E + T \{ \text{print('+')} \} \\ E &\rightarrow T \end{aligned}$$

a single terminal

<sup>7</sup>

$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow + T \{ \text{print('+')} \} R \\ R &\rightarrow \varepsilon \end{aligned}$$
<sup>8</sup>

# Semantic Analysis Revisit<sup>1</sup>

## • Implementing L-SDD: top-down/bottom-up<sup>2</sup>

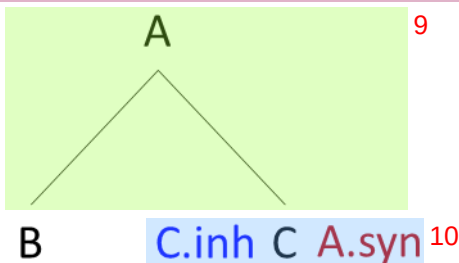
### • L-SDD $\rightarrow$ SDT:<sup>3</sup>

1. Embed the action that computes the **inherited attributes** for a nonterminal **A**<sup>4</sup> immediately **before the occurrence of A** in the production body
2. Place the actions that compute a **synthesized attribute** for the LHS **at the end of the production body**<sup>5</sup>

**A**  $\rightarrow$  B {C.inh} C {A.syn}<sup>6</sup>

• C的**继承**属性: 在C之前<sup>7</sup>

• A的**综合**属性: 在RHS末尾<sup>8</sup>



## • Eliminating left recursion!<sup>11</sup>



# Eliminating left recursion<sup>1</sup>



## • A more complex case<sup>2</sup>

$E \rightarrow E_1 + T \quad \{ E.val = E_1.val + T.val; \}$ <sup>3</sup>  
 $E \rightarrow E_1 - T \quad \{ E.val = E_1.val - T.val; \}$   
 $E \rightarrow T \quad \{ E.val = T.val; \}$   
 $T \rightarrow ( E ) \quad \{ T.val = E.val; \}$   
 $T \rightarrow \text{num} \quad \{ T.val = \text{num.val}; \}$

$E \rightarrow T R$   
 $R \rightarrow + T R_1$   
 $R \rightarrow - T R_1$   
 $R \rightarrow \epsilon$ <sup>4</sup>  
 $T \rightarrow ( E )$   
 $T \rightarrow \text{num}$

$A \rightarrow A\alpha \mid \beta$



$A \rightarrow \beta A';$ <sup>5</sup>  
 $A' \rightarrow \alpha A' \mid \epsilon$

$E \rightarrow T \{ R.i = T.val; \} R \{ E.val = R.s; \}$ <sup>6</sup>  
 $R \rightarrow + T \{ R_1.i = R.i + T.val; \} R_1 \{ R.s = R_1.s; \}$   
 $R \rightarrow - T \{ R_1.i = R.i - T.val; \} R_1 \{ R.s = R_1.s; \}$   
 $R \rightarrow \epsilon \{ R.s = R.i; \}$   
 $T \rightarrow ( E ) \{ T.val = E.val; \}$   
 $T \rightarrow \text{num} \{ T.val = \text{num.val}; \}$

解决左递归斩断了产生式 (及actions)<sup>7</sup>  
 $E \rightarrow E_1 + T \Rightarrow E \rightarrow TR \quad R \rightarrow +TR \quad R \rightarrow \epsilon$

用新引入的R, 传递属性值, 实现计算<sup>8</sup>  
 $R.i$  (属性传递)  $R.s$  (属性返回)

通过 $R \rightarrow \epsilon$ , 将计算结果传回<sup>9</sup>



# Eliminating left recursion<sup>1</sup>

$E \rightarrow T \{ R.i = T.val; \} R \{ E.val = R.s; \}$

$R \rightarrow + T \{ R_1.i = R.i + T.val; \} R_1 \{ R.s = R_1.s; \}$ <sup>2</sup>

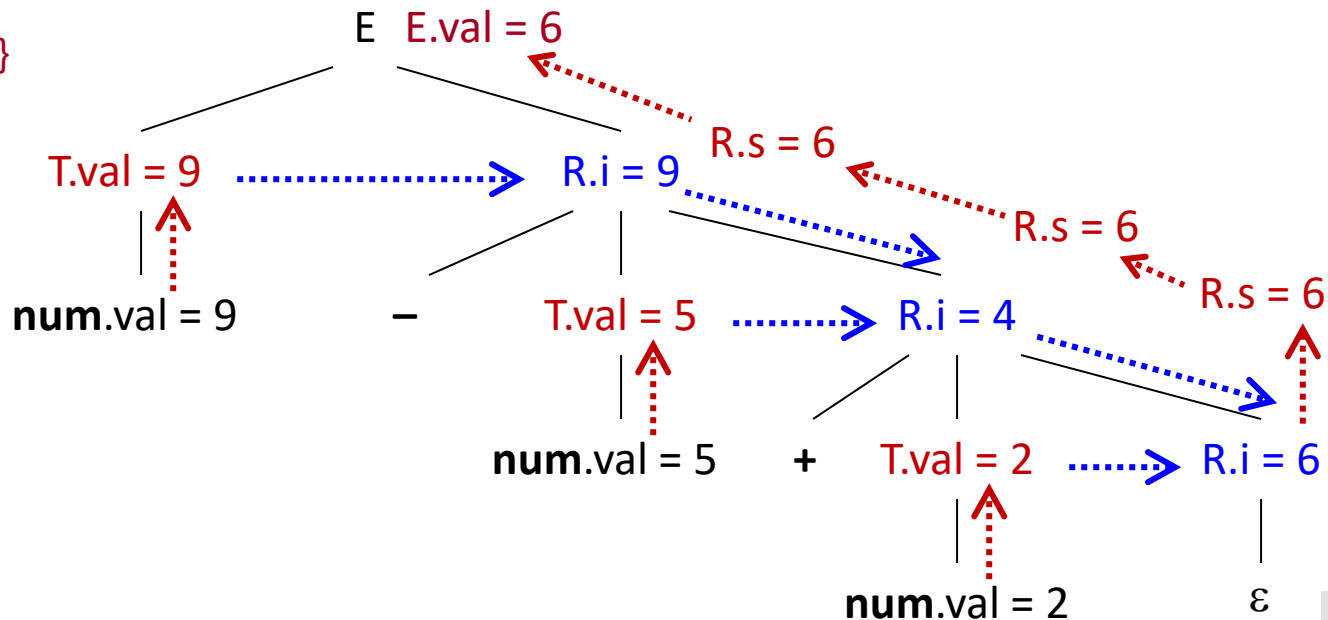
$R \rightarrow - T \{ R_1.i = R.i - T.val; \} R_1 \{ R.s = R_1.s; \}$ <sup>3</sup>

$R \rightarrow \varepsilon \{ R.s = R.i; \}$

$T \rightarrow ( E ) \{ T.val = E.val; \}$

$T \rightarrow \text{num} \{ T.val = \text{num.val}; \}$

Input: 9 - 5 + 2

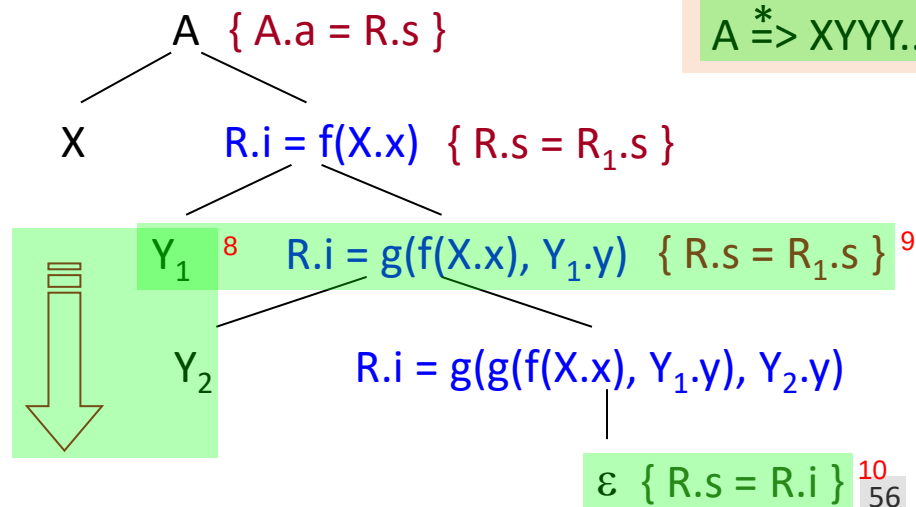
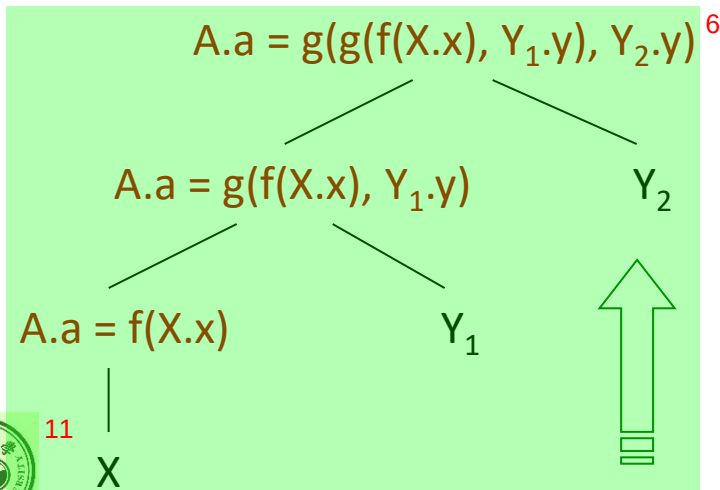


# Eliminating left recursion<sup>1</sup>

## • General rules<sup>2</sup>

- Only available for S-SDD (postfix translation schemes), a subset of L-SDD<sup>3</sup>

$$\begin{aligned} A &\rightarrow A_1 Y \{ A.a = g(A_1.a, Y.y) \} \\ A &\rightarrow X \{ A.a = f(X.x) \} \end{aligned} \quad 4$$

$$\begin{aligned} A &\rightarrow X \{ R.i = f(X.x) \} R \{ A.a = R.s \} \\ R &\rightarrow Y \{ R_1.i = g(R.i, Y.y) \} R_1 \{ R.s = R_1.s \} \\ R &\rightarrow \varepsilon \{ R.s = R.i \} \end{aligned} \quad 5$$


# Semantic Analysis Revisit<sup>1</sup>

- Evaluation order  $\rightarrow$  DAG  $\rightarrow$  topological sort  $\rightarrow$  circle is NOT allowed<sup>2</sup>

- SDD Subsets without circle: **S-SDD** and **L-SDD**<sup>3</sup>

- **S-SDD**: Synthesized attributes only<sup>4</sup>

- **L-SDD**: Synthesized and Inherited attributes ( $A \rightarrow X_1 X_2 \dots X_{i-1} X_i$ )<sup>5</sup>

- $X_i.inh$  can depend on: (i)  $A.inh$ , (ii)  $X_j.attr, j < i$ , (iii)  $X_i.attr$  with **no circles**<sup>6</sup>

- Implementation:<sup>7</sup>

- Based on parser tree or AST<sup>8</sup>

- Any computation order without dependency circles<sup>9</sup>

- Along with Parser<sup>10</sup>

- Strictly follows the parsing order: Top-down or Bottom-up<sup>11</sup>

L-SDD

S/L-SDD





# Semantic Analysis Revisit<sup>1</sup>

## • Implementing S-SDD: bottom-up<sup>2</sup>

- S-SDD  $\rightarrow$  SDT: Place each action at the end of the production<sup>3</sup>
- Slack manuscript: enable automation<sup>4</sup>

S-SDD<sup>5</sup>

| Productions                 | Semantic rules            |
|-----------------------------|---------------------------|
| (1) $L \rightarrow E$       | $print(E.val)$            |
| (2) $E \rightarrow E_1 + T$ | $E.val = E_1.val + T.val$ |
| (3) $E \rightarrow T$       | $E.val = T.val$           |
| (4) $T \rightarrow T_1 * F$ | $T.val = T_1.val * F.val$ |
| (5) $T \rightarrow F$       | $T.val = F.val$           |
| (6) $F \rightarrow (E)$     | $F.val = E.val$           |
| (7) $F \rightarrow digit$   | $F.val = digit.lexval$    |

SDT

| CFG with actions                                          |
|-----------------------------------------------------------|
| (1) $L \rightarrow E \{ print(E.val) \}$                  |
| (2) $E \rightarrow E_1 + T \{ E.val = E_1.val + T.val \}$ |
| (3) $E \rightarrow T \{ E.val = T.val \}$                 |
| (4) $T \rightarrow T_1 * F \{ T.val = T_1.val * F.val \}$ |
| (5) $T \rightarrow F \{ T.val = F.val \}$                 |
| (6) $F \rightarrow (E) \{ F.val = E.val \}$               |
| (7) $F \rightarrow digit \{ F.val = digit.lexval \}$      |

## • Example: $A \rightarrow XYZ$ <sup>7</sup>

◆  $x, y, z$  are attributes of  $X, Y, Z$  respectively<sup>8</sup>

◆ After the action,  $A$  and its attributes are at the top (i.e.,  $m-2$ )<sup>9</sup>

|            |       |     |           |           |       |
|------------|-------|-----|-----------|-----------|-------|
| State:     | $S_0$ | ... | $S_{m-2}$ | $S_{m-1}$ | $S_m$ |
| Symbol:    | $\$$  | ... | $X$       | $Y$       | $Z$   |
| Attribute: |       | ... | $X.x$     | $Y.y$     | $Z.z$ |
|            |       |     |           |           | top   |



|                         |       |     |           |
|-------------------------|-------|-----|-----------|
| state $\rightarrow$     | $S_0$ | ... | $S_{m-2}$ |
| Symbol $\rightarrow$    | $\$$  | ... | $A$       |
| Attribute $\rightarrow$ | -     | ... | $A.a$     |
|                         |       |     | top       |

# Semantic Analysis Revisit<sup>1</sup>

## • Implementing L-SDD: top-down/bottom-up<sup>2</sup>

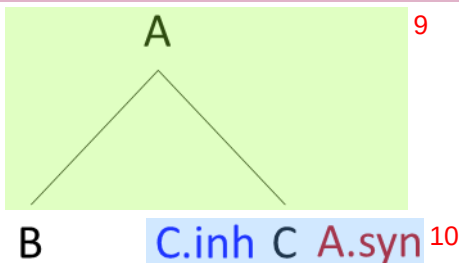
### • L-SDD $\rightarrow$ SDT:<sup>3</sup>

1. Embed the action that computes the **inherited attributes** for a nonterminal **A**<sup>4</sup> immediately **before the occurrence of A** in the production body
2. Place the actions that compute a **synthesized attribute** for the LHS **at the end of the production body**<sup>5</sup>

**A**  $\rightarrow$  B {C.inh} C {A.syn}<sup>6</sup>

• C的**继承**属性: 在C之前<sup>7</sup>

• A的**综合**属性: 在RHS末尾<sup>8</sup>



## • Eliminating left recursion!<sup>11</sup>



# Eliminating left recursion<sup>1</sup>

## • A more complex case<sup>2</sup>

$E \rightarrow E_1 + T \quad \{ E.val = E_1.val + T.val; \}$ <sup>3</sup>  
 $E \rightarrow E_1 - T \quad \{ E.val = E_1.val - T.val; \}$   
 $E \rightarrow T \quad \{ E.val = T.val; \}$   
 $T \rightarrow ( E ) \quad \{ T.val = E.val; \}$   
 $T \rightarrow \text{num} \quad \{ T.val = \text{num.val}; \}$

$E \rightarrow T R$   
 $R \rightarrow + T R_1$   
 $R \rightarrow - T R_1$   
 $R \rightarrow \epsilon$ <sup>4</sup>  
 $T \rightarrow ( E )$   
 $T \rightarrow \text{num}$

$A \rightarrow A\alpha \mid \beta$

$\Downarrow$

$A \rightarrow \beta A';$ <sup>5</sup>  
 $A' \rightarrow \alpha A' \mid \epsilon$

$E \rightarrow T \{ R.i = T.val; \} R \{ E.val = R.s; \}$ <sup>6</sup>  
 $R \rightarrow + T \{ R_1.i = R.i + T.val; \} R_1 \{ R.s = R_1.s; \}$   
 $R \rightarrow - T \{ R_1.i = R.i - T.val; \} R_1 \{ R.s = R_1.s; \}$   
 $R \rightarrow \epsilon \{ R.s = R.i; \}$   
 $T \rightarrow ( E ) \{ T.val = E.val; \}$   
 $T \rightarrow \text{num} \{ T.val = \text{num.val}; \}$

解决左递归斩断了产生式 (及actions)<sup>7</sup>  
 $E \rightarrow E_1 + T \Rightarrow E \rightarrow TR \quad R \rightarrow +TR \quad R \rightarrow \epsilon$

用新引入的R, 传递属性值, 实现计算<sup>9</sup>  
 $R.i$  (属性传递)  $R.s$  (属性返回)

通过 $R \rightarrow \epsilon$ , 将计算结果传回<sup>10</sup>



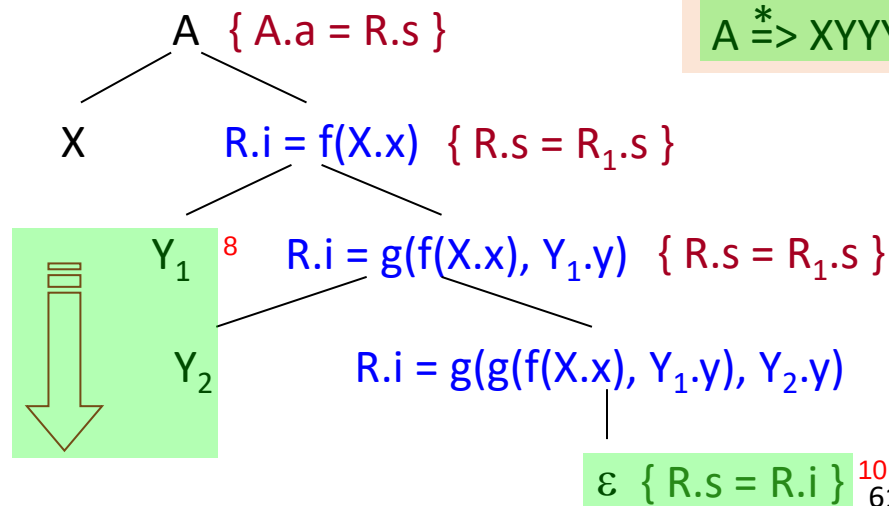
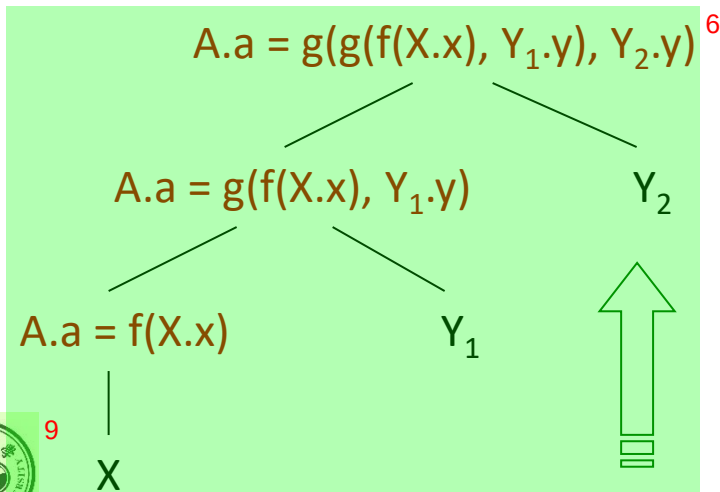
# Eliminating left recursion<sup>1</sup>

## • General rules<sup>2</sup>

- Only available for S-SDD (postfix translation schemes), a subset of L-SDD<sup>3</sup>

$A \rightarrow A_1 Y \{ A.a = g(A_1.a, Y.y) \}$ <sup>4</sup>  
 $A \rightarrow X \{ A.a = f(X.x) \}$

$A \rightarrow X \{ R.i = f(X.x) \} R \{ A.a = R.s \}$ <sup>5</sup>  
 $R \rightarrow Y \{ R_1.i = g(R.i, Y.y) \} R_1 \{ R.s = R_1.s \}$   
 $R \rightarrow \varepsilon \{ R.s = R.i \}$



$A \xRightarrow{*} XYYY\dots$ <sup>7</sup>

# L-SDD in Recursive Decent Parsing<sup>1</sup>

- A recursive-descent parser has a **function A** for each **nonterminal A**<sup>2</sup>  
[递归预测分析方法]

- ◆ Non-terminal expansion implemented by a function call<sup>3</sup>

- (Recursive) calls to functions for non-terminals in RHS<sup>4</sup>

- **Synthesized attributes**: evaluate at end of function [综合属性: 最后计算]<sup>5</sup>

- ◆ All calls for RHS would have done by then

- **Inherited attributes**: pass as argument to function [继承属性: 参数传递]<sup>6</sup>

- ◆ Values may come from parent or siblings<sup>7</sup>

- ◆ L-attributed guarantees they have been computed  
(can only come from already computed portion of RHS)

It becomes clearer here that the **inherited attributes of symbols at RHS** cannot depend on the **synthesized attribute of the LHS**<sup>9</sup>



# Example<sup>1</sup>

- Function arguments and return[参数和返回值]<sup>2</sup>

◆ Inherited: arguments<sup>3</sup>

◆ Synthesized: return<sup>4</sup>

## Augmentation:<sup>5</sup>

- Use local variables[增加局部变量]<sup>6</sup>
- Embed semantic actions[嵌入语义动作]<sup>7</sup>

### SDT<sup>8</sup>

- (1)  $T \rightarrow F$  {  $T'.inh = F.val$  }  $T'$  {  $T.val = T'.syn$  }
- (2)  $T' \rightarrow *F$  {  $T_1'.inh = T'.inh * F.val$  }  $T_1'$  {  $T'.syn = T_1'.syn$  }
- (3)  $T' \rightarrow \epsilon$  {  $T'.syn = T'.inh$  }
- (4)  $F \rightarrow digit$  {  $F.val = digit.lexval$  }

```
T'.syn T'(token, T'.inh) {  
  if token = "*", then {  
    getNext(token);  
    F.val = F(token);  
    T1'.inh = T'.inh * F.val;  
    getNext(token);  
    T1'.syn = T1'(token, T1'.inh);  
    T'.syn = T1'.syn;  
    return T'.syn;  
  } else if token = "$", then {  
    T'.syn = T'.inh;  
    return T'.syn;  
  } else  
    Error;  
}
```

D:  $F.val \rightarrow T_1'.inh \rightarrow T_1'.syn$  63

# L-SDD in LL Parsing<sup>1</sup>[非递归预测]

- **Extend the stack** to hold **actions** and certain **data items** needed for attribute evaluation[扩展语法分析栈]<sup>2</sup>

- ◆ **Action-record**[动作记录]: represent the actions to be executed<sup>3</sup>

- ◆ **Synthesize-record**[综合记录]: hold synthesized attributes for non-terminals<sup>4</sup>

- ◆ Typically, the data items are copies of attributes[属性备份]<sup>5</sup>

| Action | Code      |
|--------|-----------|
| A      | Inh Attr. |
| A.syn  | Syn Attr. |

<sup>7</sup>

- **Manage attributes on the stack**[管理属性信息]<sup>6</sup>

- ◆ The **inherited** attributes of a **nonterminal A** are placed **in the stack** that represents that terminal[A的继承属性直接放在符号位]<sup>8</sup>

- Action-record to evaluate these attributes are immediately above A<sup>9</sup>

- ◆ The **synthesized** attributes of a **nonterminal A** are placed in a **separate** **synthesize-record** that is immediately below A[A的综合属性另存放在A后]<sup>10</sup>

# L-SDD in LL Parsing (cont.)<sup>1</sup>

- Table-driven LL-parser<sup>2</sup>

- ◆ Mimics a leftmost derivation  $\rightarrow$  stack expansion<sup>3</sup>

- $A \rightarrow BC$ , suppose nonterminal  $C$  has an inherited attr  $C.i$ <sup>4</sup>

- ◆  $C.i$  may depend not only on the inherited attr  $A.i$ , but on all attrs of  $B$ <sup>5</sup>

- ◆ As an L-SDD, it ensures  $A.i$  are available when  $A$  is on the stack top<sup>6</sup>

- Thus, available to be copied into  $C$

- ◆  $A$ 's synthesized attrs  $A.s$  remain on the stack,<sup>7</sup>

- below  $B$  and  $C$  when expansion happens<sup>8</sup>

| Action | Code      |
|--------|-----------|
| A      | Inh Attr. |
| A.syn  | Syn Attr. |

| Action | Code      |
|--------|-----------|
| B      | Inh Attr. |
| B.syn  | Syn Attr. |
| C      | Inh Attr. |
| C.syn  | Syn Attr. |
| A.syn  | Syn Attr. |





# L-SDD in LL Parsing (cont.)<sup>1</sup>

- A → BC: C.i may depend not only on the inherited attr A.i, but on all the attrs of B<sup>2</sup>

- ◆ Thus, need to process B completely before C.i can be evaluated<sup>3</sup>

- ◆ Save temporary copies of all attrs needed by C.i in the action-record that evaluates C.i;<sup>4</sup>

- ◆ Otherwise, when the parser replaces A on top of the stack by BC, the A.i<sup>5</sup> will be gone, along with its stack record

1. 左部展开时 (i.e., 左部符号本身的记录出栈时), 若其含有继承属性, 则要将继承属性复制给后面的动作记录<sup>6</sup>

2. 综合属性出栈时, 要将综合属性值复制给后面的动作记录<sup>7</sup>

| Action | Code      |
|--------|-----------|
| B      | Inh Attr. |
| B.syn  | Syn Attr. |
| C      | Inh Attr. |
| C.syn  | Syn Attr. |
| A.syn  | Syn Attr. |



1

2

3

4

5

6



8

# Example (cont.)<sup>1</sup>

Input: 3\*5<sup>2</sup>

Stack top 'digit' matches the input '3'<sup>3</sup>

- pop 'digit', but value copy is needed<sup>4</sup>

(1)  $T \rightarrow F \{a_1\} T' \{a_2\}$   $a_1:T'.inh = F.val$   
 $a_2:T.val = T'.syn$   
(2)  $T' \rightarrow * F \{a_3\} T_1' \{a_4\}$   $a_3:T_1'.inh = T'.inh \times F.val$   
 $a_4:T'.syn = T_1'.syn$   
(3)  $T' \rightarrow \epsilon \{a_5\}$   $a_5:T'.syn = T'.inh$   
(4)  $F \rightarrow digit \{a_6\}$   $a_6:F.val = digit.lexval$ <sup>5</sup>

$T \rightarrow F \{a_1\} T' \{a_2\}$ <sup>6</sup>

$a_6: stack[top-1].val = stack[top].d\_lexval$ <sup>7</sup>

$F \rightarrow digit \{a_6\}$ <sup>8</sup>

|        |                   |       |                   |     |       |                   |      |    |
|--------|-------------------|-------|-------------------|-----|-------|-------------------|------|----|
| digit  | {a <sub>6</sub> } | F'syn | {a <sub>1</sub> } | T'  | T'syn | {a <sub>2</sub> } | Tsyn | \$ |
| lexv=3 | d_lexv=3          | val=3 | val=3             | inh | val   |                   | val  |    |

<sup>9</sup>



# L-SDD in LR Parsing<sup>1</sup>

## • What we already learnt<sup>2</sup>

### ◆ LR > LL, w.r.t parsing power<sup>4</sup>

□ We can do bottom-up if we can do top-down<sup>8</sup>

### ◆ S-attributed SDD can be implemented in bottom-up way<sup>9</sup>

□ All semantic actions are at the end of productions, i.e., triggered in reduce<sup>10</sup>

## • For L-SDD on an LL grammar, can it be implemented during bottom-up parsing?<sup>11</sup>

### ◆ Problem: semantic actions can be in anywhere of the production body<sup>12</sup>

(1)  $T \rightarrow F \{T'.inh = F.val\} T' \{T.val = T'.syn\}$ <sup>3</sup>  
(2)  $T' \rightarrow *F \{T_1'.inh = T'.int \times F.val\} T_1' \{T'.syn = T_1'.syn\}$ <sup>5</sup>  
(3)  $T' \rightarrow \varepsilon \{T'.syn = T'.inh\}$ <sup>6</sup>  
(4)  $F \rightarrow \text{digit} \{F.val = \text{digit.lexval}\}$ <sup>7</sup>

| Action | Code      |
|--------|-----------|
| B      | Inh Attr. |
| B.syn  | Syn Attr. |
| C      | Inh Attr. |
| C.syn  | Syn Attr. |
| A.syn  | Syn Attr. |



# L-SDD in LR - The Problem<sup>1</sup>

- It is not natural to evaluate inherited attributes<sup>2</sup>

- ◆ Example: how to get  $T'.inh$ <sup>3</sup>

- Claims: inherited attributes are on the stack<sup>4</sup>

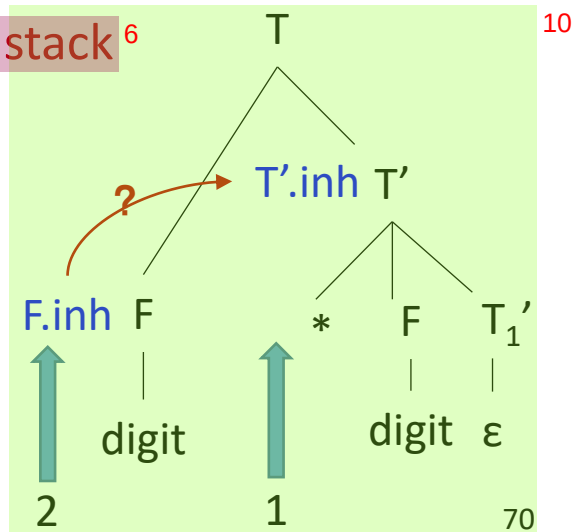
- ◆ L-SDD guarantee they've already been computed<sup>5</sup>

- ◆ But computed by previous productions - **deep in the stack**<sup>6</sup>

## • Solution<sup>7</sup>

- ◆ **Hack the stack to dig out those values**<sup>8</sup>

- (1)  $T \rightarrow F$   $\{T'.inh = F.val\}$   $T' \{T.val = T'.syn\}$
  - (2)  $T' \rightarrow *F$   $\{T_1'.inh = T'.int \times F.val\}$   $T_1' \{T'.syn = T_1'.syn\}$
  - (3)  $T' \rightarrow \epsilon$   $\{T'.syn = T'.inh\}$
  - (4)  $F \rightarrow digit$   $\{F.val = digit.lexval\}$



# L-SDD in LR - Marker<sup>1</sup>

- Given the following SDD, where  $|\alpha| \neq |\beta|$ <sup>2</sup>

- ◆  $A \rightarrow X\alpha \{Y.in = X.s\} Y \mid X\beta \{Y.in = X.s\} Y$ <sup>3</sup>

- ◆  $Y \rightarrow \gamma \{Y.s = f(Y.in)\}$ <sup>4</sup>

- Problem: cannot generate stack location for  $Y.in$ <sup>5</sup>

- ◆ Because  $X.s$  is at different **relative locations** from  $Y$  in the stack<sup>6</sup>

- Solution: insert **markers**  $M_1, M_2$  right before  $Y$ <sup>7</sup>

- ◆  $A \rightarrow X\alpha M_1Y \mid X\beta M_2Y$ <sup>8</sup>

- ◆  $M_1 \rightarrow \epsilon \{M_1.s = \text{stack}[\text{top} - |\alpha|].s\} \quad // \quad M1.s = X.s$ <sup>9</sup>

- ◆  $M_2 \rightarrow \epsilon \{M_2.s = \text{stack}[\text{top} - |\beta|].s\} \quad // \quad M2.s = X.s$ <sup>10</sup>

- ◆  $Y \rightarrow \gamma \{Y.s = f(\text{stack}[\text{top} - |\gamma|].s)\} \quad // \quad Y.s = M1.s \text{ or } Y.s = M2.s$ <sup>11</sup>

- Marker: a **non-terminal** marking a location that is the same with the symbol that has an inherited attribute<sup>12</sup>

- ◆ Always produces  $\epsilon$  since it is only a placeholder for an action<sup>14</sup>

# Modify Grammar with Marker<sup>1</sup>

- Given an L-SDD on an LL grammar, we can adapt the grammar to compute the same SDD during an LR parse<sup>2</sup>
  - Put a **marker non-terminal** [标记非终结符] in the place of each embedded action<sup>3</sup>
    - Each of such places gets a distinct marker, say  $M$ , where  $M \rightarrow \epsilon$  [空产生式]
  - Modify the action  $\{a\}$  if marker  $M$  replaces it in some production  $A \rightarrow \alpha \{a\} \beta$ ,<sup>4</sup> and associate with  $M \rightarrow \epsilon$  an action  $\{a'\}$  that
    - Copies, as inherited attrs of  $M$ , any attrs of  $A$  or symbols of  $\alpha$  that action  $\{a\}$  needs (e.g.,<sup>5</sup>  $M.i = A.i$ ) [M的继承属性为{a}的输入]
    - Compute sattrs in the same way as  $\{a\}$ , but makes those attrs be synthesized attrs of  $M$  (e.g.,<sup>6</sup>  $M.s = f(M.i)$ ) [M的综合属性为{a}中的输出]

$A \rightarrow \{B.i = f(A.i)\} BC$ <sup>7</sup>

$A \rightarrow M BC$ <sup>8</sup>  
 $M \rightarrow \epsilon \{M.i = A.i; M.s = f(M.i)\}$

# Example<sup>1</sup>

(1)  $T \rightarrow F \{T'.inh = F.val\} T' \{T.val = T'.syn\}$   
(2)  $T' \rightarrow *F \{T_1'.inh = T'.int * F.val\} T_1' \{T'.syn = T_1'.syn\}$   
(3)  $T' \rightarrow \varepsilon \{T'.syn = T'.inh\}$   
(4)  $F \rightarrow digit \{F.val = digit.lexval\}$



(1)  $T \rightarrow F \mathbf{M} T' \{T.val = T'.syn\}$   
 $\mathbf{M} \rightarrow \varepsilon \{\mathbf{M.i} = F.val; \mathbf{M.s} = \mathbf{M.i};\}$   
(2)  $T' \rightarrow *F \mathbf{N} T_1' \{T'.syn = T_1'.syn\}$   
 $\mathbf{N} \rightarrow \varepsilon \{\mathbf{N.i1} = T.inh; \mathbf{N.i2} = F.val; \mathbf{N.s} = \mathbf{N.i1} * \mathbf{N.i2}\}$   
(3)  $T' \rightarrow \varepsilon \{T'.syn = T'.inh\}$   
(4)  $F \rightarrow digit \{F.val = digit.lexval\}$

$A \rightarrow \{B.i = f(A.i)\} BC$



$A \rightarrow \mathbf{M} B C$

$M \rightarrow \varepsilon \{M.i = A.i; M.s = f(M.i)\}$





# Stack Manipulation[栈操作]<sup>1</sup>



(1)  $T \rightarrow F \{T'.inh = F.val\} T' \{T.val = T'.syn\}$   
(2)  $T' \rightarrow *F \{T_1'.inh = T'.int \times F.val\} T_1' \{T'.syn = T_1'.syn\}$   
(3)  $T' \rightarrow \varepsilon \{T'.syn = T'.inh\}$   
(4)  $F \rightarrow digit \{F.val = digit.lexval\}$

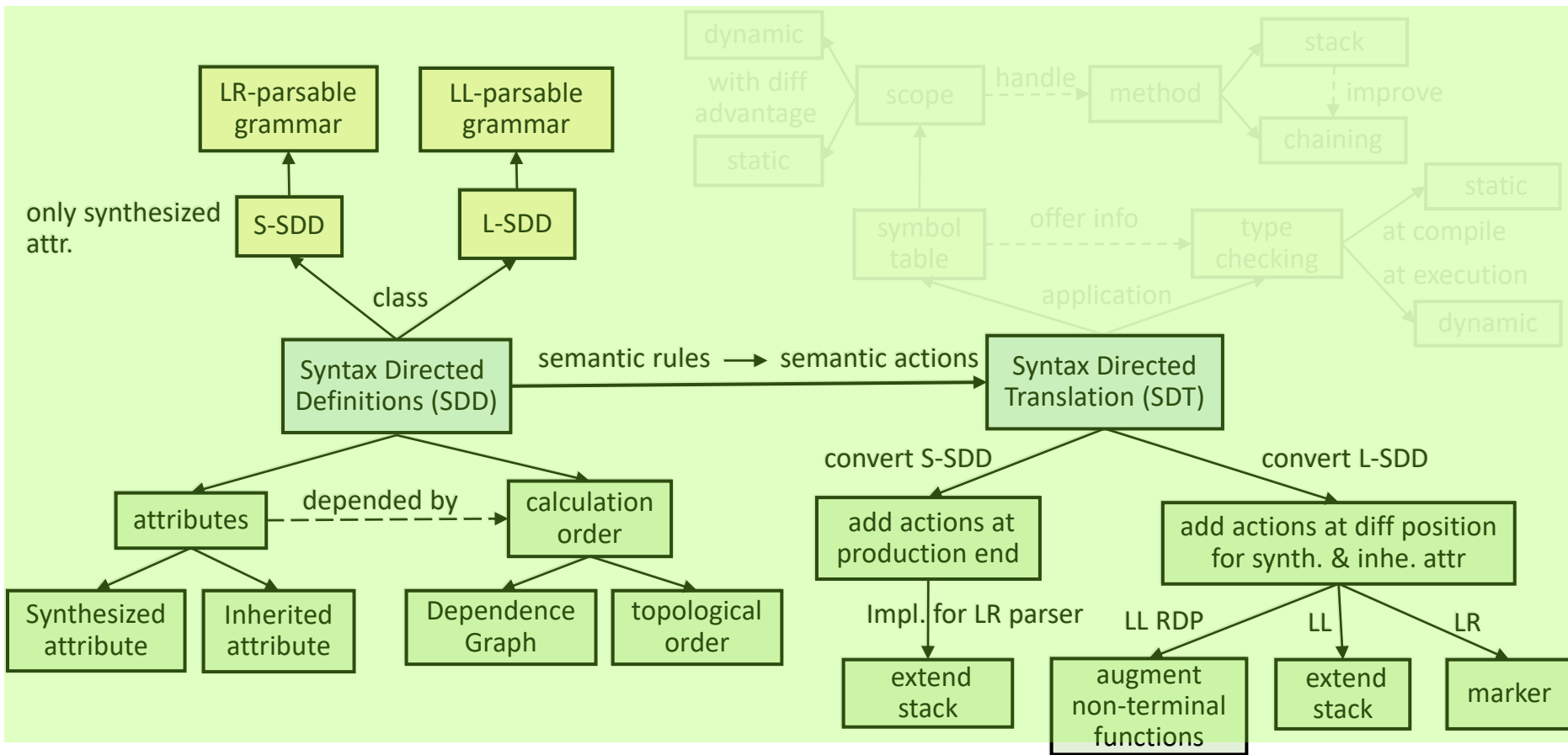
(1)  $T \rightarrow F M T' \{T.val = T'.syn\}$   
     $M \rightarrow \varepsilon \{M.i = F.val; M.s = M.i;\}$   
(2)  $T' \rightarrow *F N T_1' \{T'.syn = T_1'.syn\}$   
     $N \rightarrow \varepsilon \{N.i1 = T.inh; N.i2 = F.val; N.s = N.i1 * N.i2\}$   
(3)  $T' \rightarrow \varepsilon \{T'.syn = T'.inh\}$   
(4)  $F \rightarrow digit \{F.val = digit.lexval\}$



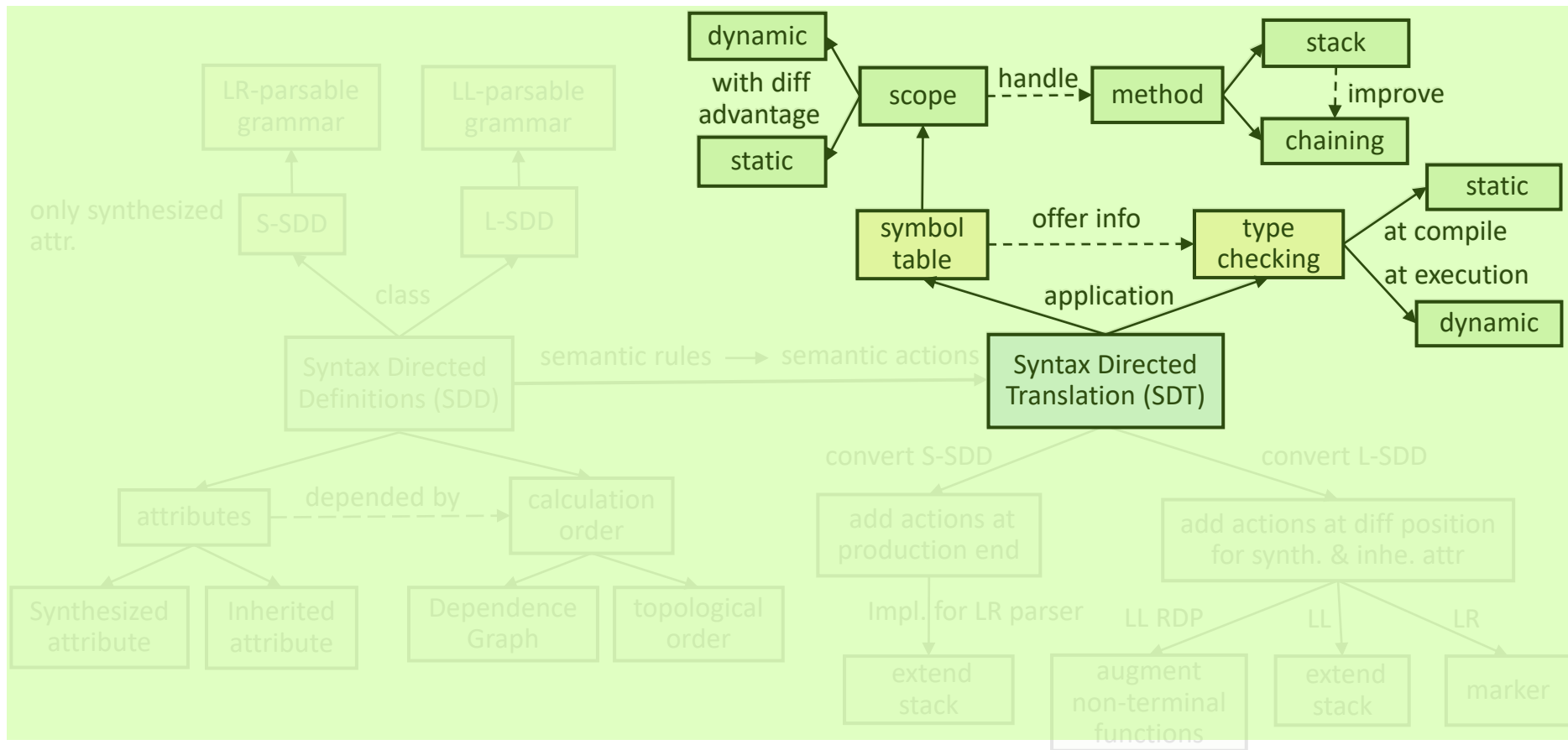
(1)  $T \rightarrow F M T' \{stack[top-2].val = stack[top].syn; top = top - 2;\}$   
     $M \rightarrow \varepsilon \{stack[top+1].T'.inh = stack[top].val; top = top + 1;\}$   
(2)  $T' \rightarrow *F N T_1' \{stack[top-3].syn = stack[top].syn; top = top - 3;\}$   
     $N \rightarrow \varepsilon \{stack[top+1].T'.inh = stack[top-2].T'.inh * stack[top].val; top = top + 1;\}$   
(3)  $T' \rightarrow \varepsilon \{stack[top+1].syn = stack[top].T'.inh; top = top + 1;\}$   
(4)  $F \rightarrow digit \{stack[top].val = stack[top].lexval;\}$



# Content<sup>1</sup>



# Content<sup>1</sup>



# Binding [绑定]<sup>1</sup>

- **Binding**: matching identifier use with definition[使用-定义]<sup>2</sup>
  - ◆ Definition: associating an id with a memory location<sup>3</sup>
  - ◆ Binding is an essential step before machine code generation<sup>4</sup>
- If there are multiple definitions, which one to use?<sup>5</sup>

```
void foo()  
{  
    char x; /* allocated at mem[0x100] */  
    ...  
    {  
        int x; /* allocated at mem[0x200] */  
        ...  
    }  
    x = x + 1; /* add mem[0x100], 1 ? add mem[0x200], 1 ?  
}
```



# Scope [作用域]<sup>1</sup>

- **Scope**: **program region** where a definition can be bound<sup>2</sup>

- ◆ Uses of identifier in the scope is bound to that definition<sup>3</sup>

- ◆ For Java: private, global, static, etc...<sup>4</sup>

- Some properties of scopes<sup>5</sup>

- ◆ Use not in scope of any definition results in **undefined error**<sup>6</sup>

- ◆ Scopes for the same identifier can **never overlap**<sup>7</sup>

- There is **at most one binding** at any given time

- Two types: static scoping and dynamic scoping [静态/动态作用域]<sup>8</sup>

- ◆ Depending on how scopes are formed

# Static Scoping [静态作用域]<sup>1</sup>

- Scopes formed by where definitions are in **program text** [声明起作用的那段区域]<sup>2</sup>
  - ◆ Also known as **lexical scoping** since related to program text C/C++, Java, Python, JavaScript [也叫词法作用域]<sup>3</sup>
- Rule: bind to the **closest enclosing definition**<sup>4</sup>

```
void foo()  
{  
    char x;  
    ...  
    {  
        int x;  
        ...  
    }  
    x = x + 1;  
}
```



# Dynamic Scoping<sup>1</sup>[动态作用域]

- Scopes formed by when definitions happen **during runtime**<sup>2</sup>[运行时决定]

◆ Perl, Bash, LISP, Scheme<sup>3</sup>

- Rule: bind to **most recent definition** in current execution<sup>4</sup>

```
void foo()  
{  
    (1) char x;  
    (2) if (...) {  
        (3) int x;  
        (4) x=x+1;  
    }  
    (5) x='a';  
}
```

- Which x's definition is the most recent?<sup>6</sup>

◆ Execution (a): ...**(1)**...(2)...(5)

◆ Execution (b): ...(1)...(2)...**(3)**...(4)...(5)

# Static vs. Dynamic Scoping<sup>1</sup>[对比]

- Most languages that started with dynamic scoping (LISP, Scheme, Perl) added static scoping afterwards<sup>2</sup>
- Why? With **dynamic scoping**...<sup>3</sup>
  - ◆ All bindings are done at execution time<sup>4</sup>
  - ◆ Hard to figure out by eyeballing, for both compiler and human<sup>5</sup>
- Pros of **static scoping**[静态的好处]<sup>6</sup>
  - ◆ Static scoping leads to **fewer programmer errors**<sup>7</sup>
    - Bindings readily apparent from lexical structure of code<sup>8</sup>
  - ◆ Static scoping leads to **more efficient code**<sup>9</sup>
    - Can determine bindings at compile time – allow code optimization<sup>10</sup>



# What is Symbol Table[符号表]<sup>1</sup>

- **Symbol**: same thing as **identifier** (used interchangeably)<sup>2</sup>
- **Symbol table**: a data structure that tracks info about all symbols<sup>3</sup>
  - ◆ Each entry represents a definition of that identifier[标识符的定义]<sup>4</sup>
  - ◆ Maintains definitions that reach current program point[当前作用域中的定义]<sup>5</sup>
  - ◆ List updated whenever scopes are entered or exited [随作用域改变而更新]<sup>6</sup>
  - ◆ Used to perform binding of identifier uses [用于绑定变量定义与使用]<sup>7</sup>
    - Traversing the parse tree in a separate pass after parsing [单独扫描语法树]<sup>8</sup>
    - Using semantic actions as an integral part of parsing pass [或与语义分析同时]<sup>9</sup>
  - ◆ Usually discarded after generating executable binary<sup>10</sup>
    - Machine code instructions no longer contain symbols<sup>11</sup>
  - ◆ For use in debuggers, symbol tables may be included<sup>12</sup>
    - To display symbol names instead of addresses in debuggers<sup>13</sup>
    - For GCC, using `'gcc -g'` includes debug symbol tables<sup>14</sup>

# Maintaining Symbol Table<sup>1</sup><sub>[维护]</sub>

- Basic idea:<sup>2</sup>

```
int x=0; ... void foo() { int x=0; ... x=x+1; } ... x=x+1 ...3
```

- ◆ Start processing *foo*:<sup>4</sup>

- ▢ Add definition of *x*, overriding old definition of *x* if any<sup>5</sup>

- ◆ After processing *foo*:<sup>6</sup>

- ▢ Remove definition of *x*, restoring old definition of *x* if any<sup>7</sup>

- Operations:<sup>8</sup>

- ◆ *enter\_scope()* start a new scope<sup>9</sup>

- ◆ *exit\_scope()* exit current scope<sup>10</sup>

- ◆ *find\_symbol(x)* find the information about *x*<sup>11</sup>

- ◆ *add\_symbol(x)* add a symbol *x* to the symbol table<sup>12</sup>

- ◆ *check\_symbol(x)* true if *x* is defined in current scope<sup>13</sup>

# Symbol Table Structure [结构]<sup>1</sup>

- Front-end time is affected by symbol table access time [符号表访问时间影响编译前端性能]<sup>2</sup>

- ◆ Front-end: lexical, syntax, semantic analysis<sup>3</sup>
- ◆ Frequent searches on any large data structure is expensive<sup>4</sup>
- ◆ Symbol table design is important for compiler performance<sup>5</sup>

- What data structure to choose? [可选数据结构]<sup>6</sup>

- ◆ List [线性表]<sup>7</sup>
- ◆ Binary tree [二叉树]<sup>8</sup>
- ◆ Hash table [哈希表]<sup>9</sup>

- Tradeoffs: time vs. space [空间和时间的权衡]<sup>10</sup>

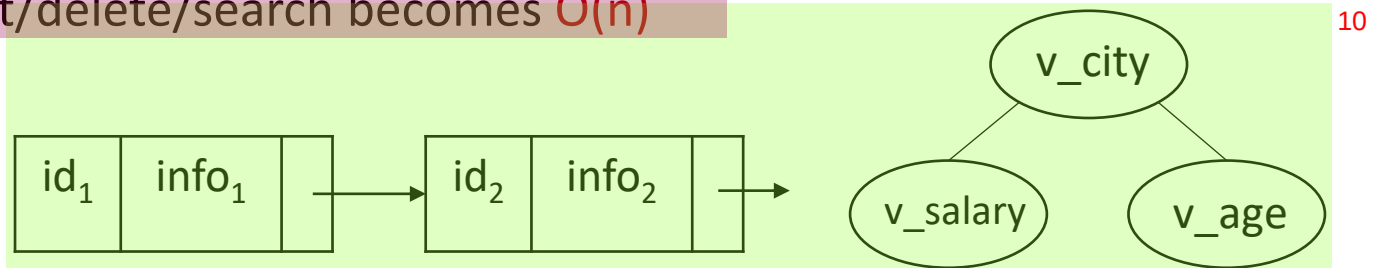
- ◆ Let us first consider the organization without scope<sup>11</sup>

# Symbol Table Structure (cont.)<sup>1</sup>

- **Array**: no space wasted, insert/delete:  $O(n)$ , search:  $O(n)$ <sup>2</sup>
- **Linked list**: extra pointer space, insert/delete:  $O(1)$ , search:  $O(n)$ <sup>3</sup>
  - ◆ Optimization: move recently used identifier to the head<sup>4</sup>
  - ◆ Frequently used identifiers are found more quickly<sup>5</sup>
- **Binary tree**: use more space than array/list<sup>6</sup>
  - ◆ But insert/delete/search is  $O(\log n)$  on balanced tree<sup>7</sup>
  - ◆ In the worst case, tree may reduce to linked list<sup>8</sup>
    - Then insert/delete/search becomes  $O(n)$

<sup>9</sup>

|        |          |
|--------|----------|
| $id_1$ | $info_1$ |
| $id_2$ | $info_2$ |
|        |          |



# SDD Implementation Revisit<sup>1</sup>

- S-SDD & LR: Slack manipulation<sup>2</sup>

- L-SDD: Remove Left Recursion<sup>3</sup>

- L-SDD & RDP: Extend RDP Function + Slack manipulation<sup>4</sup>

- ◆ **Inherited**: arguments; **Synthesized**: return<sup>5</sup>

- ◆ Use local variables[增加局部变量]<sup>6</sup>

- ◆ Embed semantic actions[嵌入语义动作]<sup>7</sup>

- ◆ Inherited: along with the symbol; Synthesized: right below the symbol<sup>10</sup>

1. 左部展开时 (i.e., 左部符号本身的记录出栈时), 若其含有**继承**属性, 则要将**继承属性复制**给后面的动作记录<sup>8</sup>

2. 综合属性出栈时, 要将**综合属性值复制**给后面的动作记录<sup>9</sup>

- L-SDD & LL(1): Slack manipulation<sup>11</sup>

- L-SDD & LR: Marker + Slack manipulation<sup>12</sup>

$A \rightarrow X\alpha \{Y.in = X.s\} Y \mid X\beta \{Y.in = X.s\} Y$   
 $Y \rightarrow \gamma \{Y.s = f(Y.in)\}$

$A \rightarrow X\alpha M_1 Y \mid X\beta M_2 Y$

$M_1 \rightarrow \epsilon \{M_1.s = \text{stack}[\text{top} - |\alpha|.s] \} \quad // \quad M_1.s = X.s$

$M_2 \rightarrow \epsilon \{M_2.s = \text{stack}[\text{top} - |\beta|.s] \} \quad // \quad M_2.s = X.s$

$Y \rightarrow \gamma \{Y.s = f(\text{stack}[\text{top} - |\gamma|.s]) \} \quad // \quad Y.s = M_1.s \text{ or } Y.s = M_2.s$

$A \rightarrow \{B.i = f(A.i)\} BC$   
↓  
 $A \rightarrow M BC$   
 $M \rightarrow \epsilon \{M.i = A.i; M.s = f(M.i)\}$ <sup>13</sup>

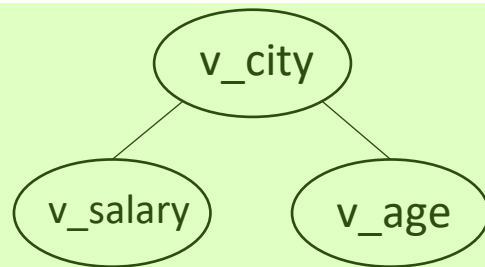
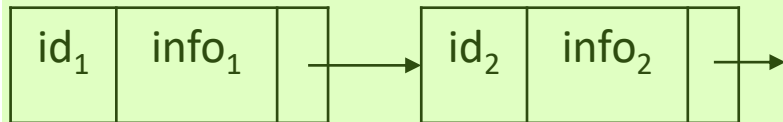


# Symbol Table Revisit<sup>1</sup>

- **Binding:** matching **identifier use** with **definition**[使用-定义]<sup>2</sup>
- **Scoping:**<sup>3</sup>
  - **Static:** Can be obtained by static code analysis<sup>4</sup>
  - **Dynamic:** becomes clear only during execution<sup>5</sup>
- **Symbol:** same thing as **identifier**<sup>6</sup>
- **Symbol table:** a **data structure** that tracks info about all symbols<sup>7</sup>
- **Constructing Symbol table:** a tradeoff between **time** and **space**<sup>8</sup>
  - Array, Linked list, Binary tree, **Hash table**<sup>9</sup>

|                 |                   |
|-----------------|-------------------|
| id <sub>1</sub> | info <sub>1</sub> |
| id <sub>2</sub> | info <sub>2</sub> |
|                 |                   |

<sup>10</sup>



<sup>11</sup>

# Symbol Table Structure (cont.)<sup>1</sup>

- $\text{hash}(\text{id\_name}) \rightarrow \text{index}$ [哈希表]<sup>2</sup>

- ◆ A **hash function** decides **mapping** from identifier to index<sup>3</sup>

- ◆ Conflicts resolved by chaining multiple IDs to same index<sup>4</sup>

- Memory consumption from hash table ( $N \ll M$ )<sup>5</sup>

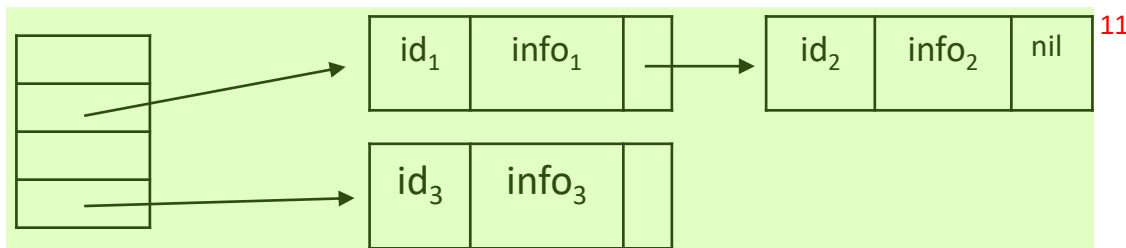
- ◆ M: the size of hash table<sup>6</sup>

- ◆ N: the number of stored identifiers<sup>7</sup>

- But insert/delete/search in  $O(1)$  time<sup>8</sup>

- ◆ Can become  $O(n)$  with frequent conflicts and long chains<sup>9</sup>

- Most compilers choose hash table for its quick access time<sup>10</sup>



# Adding Scope to Symbol Table[作用域]<sup>1</sup>

- To handle **multiple scopes** in a program [处理多个作用域]<sup>2</sup>

- ◆ Conceptually, need **an individual table** for **each scope**<sup>3</sup>

- In order to be able to **enter and exit scopes**

- ◆ Sometimes symbols in scope can be discarded on exit:<sup>4</sup>

```
if (...) { int v; } /* block scope */5  
/* v is no longer valid */
```

- ◆ Sometimes not:<sup>6</sup>

```
class X { ... void foo() {...} ... } /* class scope */7  
/* foo() is no longer valid */  
X v;  
call v.foo(); /* v.foo() is still valid */
```

- How can scoping be enforced without discarding symbols?<sup>8</sup>

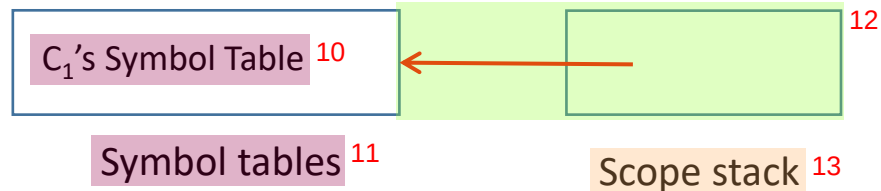
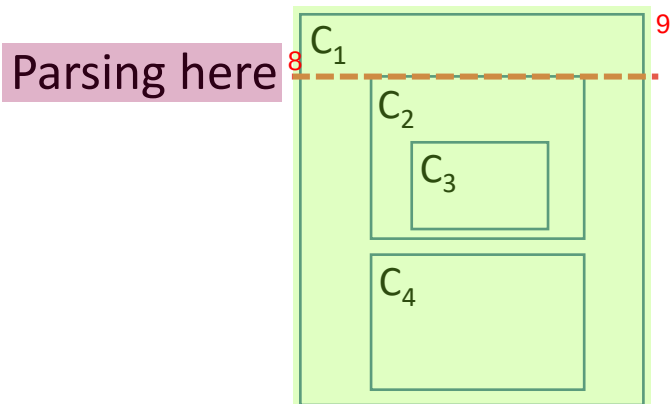
- ◆ Keep **a stack of active scopes** at a given point<sup>9</sup>

- ◆ Keep **a list of all reachable scopes** in the entire program<sup>10</sup>



# Handle Scopes with Stack<sup>1</sup>

- Organize all symbol tables into **a scope stack**[作用域栈]<sup>2</sup>
  - ◆ An individual symbol table for each scope<sup>3</sup>
    - Scope is defined by **nested lexical structure**, e.g.,  $\{C_1 \{C_2 \{C_3\}\} \{C_4\}\}$ <sup>4</sup>
  - ◆ Stack holds one entry for each open scope<sup>5</sup>
    - **Inner-most** scope is stored at the **top of the stack**<sup>6</sup>
- Stack **push/pop** happen when **entering/exiting** a scope<sup>7</sup>



# Handle Scopes with Stack (cont.)<sup>1</sup>

## • Operations:<sup>2</sup>

### ◆ When entering a scope<sup>3</sup>

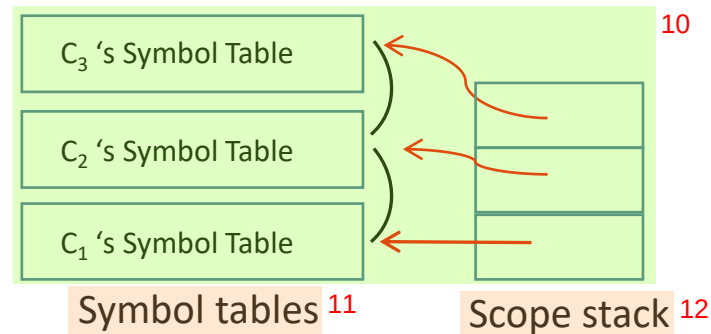
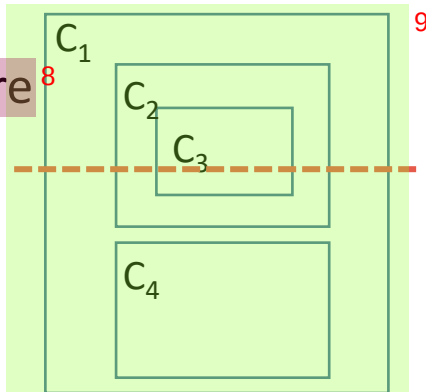
□ Create a new symbol table to hold all variables declared in that scope<sup>4</sup>

□ Push a pointer to the symbol table on the stack<sup>5</sup>

### ◆ Pop the pointer to the symbol table when exiting scope<sup>6</sup>

### ◆ Search from the top of the stack<sup>7</sup>

Parsing here<sup>8</sup>



# Info Stored in Symbol Table<sup>1</sup>

- Entry in symbol table<sup>2</sup>

- ◆ **String**: the name of identifier<sup>3</sup>

- ◆ **Kind**: variable, function, struct type, class type<sup>4</sup>

|        |      |            |
|--------|------|------------|
| string | kind | attributes |
|--------|------|------------|

<sup>5</sup>

- Attributes vary with the kind of symbols<sup>6</sup>

- ◆ **variable**: type, address of variable<sup>7</sup>

- ◆ **function**: prototype, address of function body<sup>8</sup>

- ◆ **struct type**: field names, field types<sup>9</sup>

- ◆ **class type**: symbol table for class<sup>10</sup>

# Attribute List in Symbol Table<sup>1</sup>

- Type info can be arbitrarily complicated<sup>2</sup>

- ◆ Type can be an array with multiple dimensions<sup>3</sup>

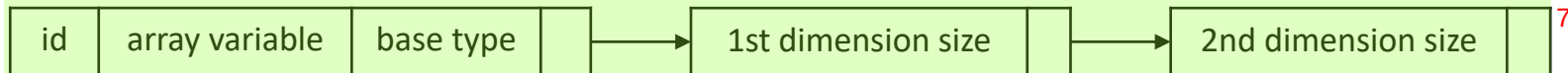
```
char arr[20][20];
```

- ◆ Type can be a struct with multiple fields<sup>4</sup>

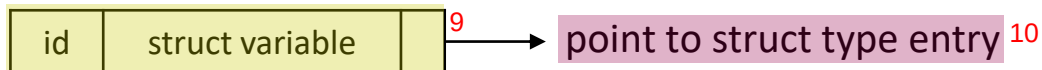
```
struct Point {  
    float x;  
    float y;  
} point;
```

- Store all type info in an **attribute list**<sup>5</sup>

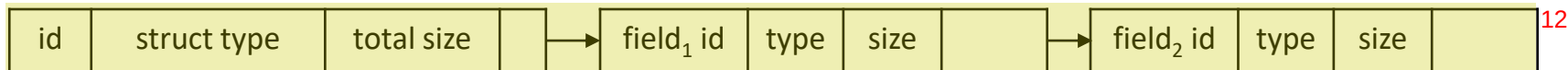
- ◆ Entry for an array variable with 2 dimensions<sup>6</sup>



- ◆ Entry for a struct variable<sup>8</sup>



- ◆ Entry for a struct type with 2 fields<sup>11</sup>



# Use Type Information[类型信息]<sup>1</sup>

- Each variable or function entry contains **type** info<sup>2</sup>
- Type info is used in later **code generation** stage[代码生成]<sup>3</sup>
  - ◆ To calculate how much memory to alloc for a variable<sup>4</sup>
    - Should a variable assignment be a **4 byte** or **8 byte** copy?
  - ◆ To translate uses of variables to machine instructions<sup>5</sup>
    - Should a '+' on variable be an **integer** or a **floating** point add?
  - ◆ To translate calls to functions to machine instructions<sup>6</sup>
    - What are the **types of arguments** and **return value** of the function?
- Also used in later **code optimization** stage[代码优化]<sup>7</sup>
  - ◆ To help compiler understand semantics of program<sup>8</sup>
- Also used in **semantic analysis** stage for **Type Checking**<sup>9</sup>
  - ◆ Uses types to check semantic correctness of program<sup>10</sup>

# Type and Type Checking<sup>1</sup>

- **Type**: a set of values and a set of operations on these values<sup>2</sup>

- **Type checking**: verifying type consistency across program<sup>3</sup>

- ◆ A program is type consistent if all operators are consistent with the operand value types<sup>4</sup>

- ◆ Much of what we do in semantic analysis is type checking<sup>5</sup>

- Some type checking examples:<sup>6</sup>

- ◆ Given `char *str = "Hello";`<sup>7</sup>

- `str[2]` is consistent: `char*` type allows `[]` operator<sup>8</sup>

- `str/2` is not: `char*` type does not allow `/` operator<sup>9</sup>

- ◆ Given `int pi = 3;`<sup>10</sup>

- `pi/2` is consistent: `int` type allows `/` operator<sup>11</sup>

- `pi=3.14` is not: `=` operator not allowed on different types<sup>12</sup>

- Compiler must **type convert implicitly** to make it consistent<sup>13</sup>

# Static Type Checking<sup>1</sup>[静态类型检查]

- **Static type checking** at compile time<sup>2</sup>

- ◆ **Infers**[推断] whether the program is type consistent through code analysis<sup>3</sup>

- Collect info via declarations that store in symbol table<sup>4</sup>

- Check the types involved in each operation<sup>5</sup>

- E.g., `int a, b, c; a = b + c;` can be proven type consistent because the addition of two *ints* is still an *int*<sup>6</sup>

- Difficult for a language to only do static type checking<sup>7</sup>

- ◆ Some type errors usually cannot be detected at compile time<sup>8</sup>

- E.g., `a` and `b` are of type *int*, `a * b` may not in the **valid range** of *int*<sup>9</sup>

- **Typecasting** can be a pretty risky thing to do - Basically, typecast suspends type checking<sup>10</sup>

- `unsigned a; (int) a;`

# Dynamic Type Checking<sup>1</sup>[动态检查]

- **Dynamic type checking** at execution time<sup>2</sup>

- ◆ Type consistency by checking types of **runtime values**<sup>3</sup>

- ◆ Include type info for each data location at runtime<sup>4</sup>

- E.g., a variable of type double would contain both the **actual double value** and some kind of **tag indicating “double type”**<sup>5</sup>

- The execution of any operation begins by first checking these type tags<sup>6</sup>

- The operation is performed only if everything checks out<sup>7</sup>

- Otherwise, a type error occurs and usually halts execution<sup>8</sup>

- ◆ Array bounds check:<sup>9</sup>

- Is `int A[10], i; ... A[i] = i;` type consistent?<sup>10</sup>

- Static type checking is always more desirable. Why?<sup>11</sup>

- ◆ Always good to catch more errors before runtime<sup>12</sup>

- ◆ Dynamic type checking carries runtime overhead<sup>13</sup>



# Static vs. Dynamic Typing<sup>1</sup>[静态-动态]

- Static typing: C/C++, Java, ...<sup>2</sup>

- ◆ Variables have **static types** → hold only one type of value<sup>3</sup>

- E.g. `int x;` → x can only hold ints<sup>4</sup>

- E.g. `char *x;` → x can only hold char pointers<sup>5</sup>

- ◆ How are types assigned to variables?<sup>6</sup>

- C/C++, Java: types are explicitly defined<sup>7</sup>

- `int x;` → explicit assignment of type int to x<sup>8</sup>

- Pros/cons of static typing<sup>9</sup>

- ◆ More programmer effort<sup>10</sup>

- Programmer must adhere to strict type rules<sup>11</sup>

- Defining advanced types can be quite complex (e.g. classes)<sup>12</sup>

- ◆ Less bugs and execution time<sup>13</sup>

- Thanks to static type checking<sup>14</sup>

# Static vs. Dynamic Typing (cont.)<sup>1</sup>

- Dynamic Typing: Python, JavaScript, PHP, ...<sup>2</sup>

- ◆ Variables have dynamic types → can hold multiple types<sup>3</sup>

- `var x; /* var declaration without a static type */`<sup>4</sup>

- `x = 1; /* now x holds an integer value */`<sup>5</sup>

- `x = "one"; /* now x holds a string value */`<sup>6</sup>

- ◆ How are types assigned to variables?<sup>7</sup>

- Type is a runtime property → type tags stored with values<sup>8</sup>

- Dynamic type checking must be done during runtime<sup>9</sup>

- Pros/cons of dynamic typing<sup>10</sup>

- ◆ Less programmer effort<sup>11</sup>

- Flexible type rule means program is **more malleable**[可塑性强]<sup>12</sup>

- Absence of types declarations means **shorter code**<sup>13</sup>

- Suitable for scripting or prototyping languages<sup>14</sup>

- ◆ More program bugs and execution time<sup>15</sup>

- Due to **dynamic type checking**<sup>16</sup>

# Type System[类型系统]<sup>1</sup>

- Static/dynamic typing are **type systems**<sup>2</sup>

- ◆ Type System: **types** and **type rules** of a language<sup>3</sup>

- Static/dynamic type checking are **methods**<sup>4</sup>

- ◆ Methods to **enforce the rules** of the given type system<sup>5</sup>

- Static type checking is not used only for static typing<sup>6</sup>

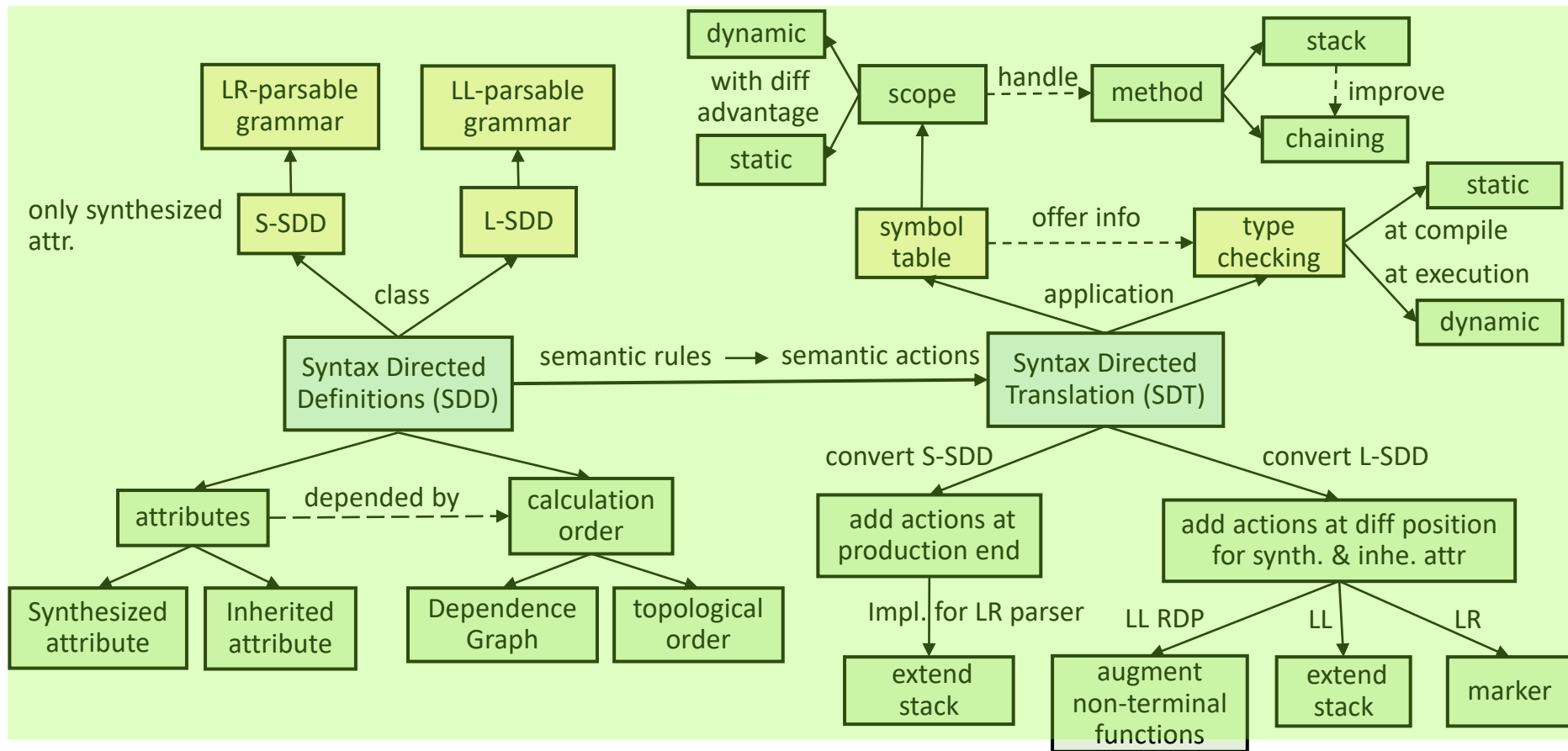
- ◆ Also used for dynamic typing **if types can be inferred** at compile time

- Dynamic type checking is not used only for dynamic typing<sup>7</sup>

- ◆ Some features of statically typed languages require it

- e.g., **downcasting**[父类向子类的强制类型转换]

# Summary<sup>1</sup>



# Summary<sup>1</sup>

- SDD & SDT<sup>2</sup>

- Synthesized & inherited attribute<sup>3</sup>

- Evaluation order – DAG & topological order<sup>4</sup>

- S-SDD & L-SDD<sup>5</sup>

- S-SDD -> SDT -> Implementation<sup>8</sup>

- (LR only) Postfix SDT -> stack extension -> stack manipulation<sup>9</sup>

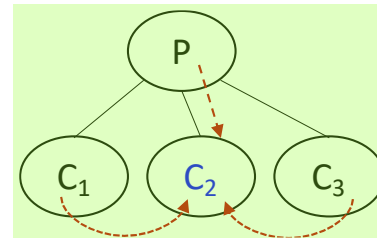
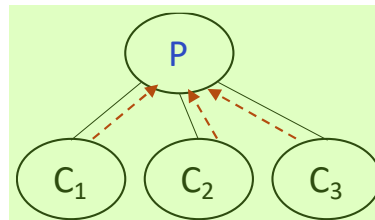
- L-SDD -> SDT -> Implementation<sup>10</sup>

- LL: Eliminate left recursion<sup>11</sup>

- RDP: inherited attributes (arguments) and synthesized attributes (return values)<sup>12</sup>

- Predictive Parsing: stack extension (action and synthesized records)<sup>14</sup>

- LR: Marker -> postfix SDT -> stack extension -> stack manipulation<sup>15</sup>



# Summary<sup>1</sup>

- Static and Dynamic Scoping<sup>2</sup>

- ◆ Pros of **static**: fewer errors + more efficient<sup>3</sup>

- Symbol and Symbol Table<sup>4</sup>

- ◆ Structuring: array, list, binary tree, **hash table**<sup>5</sup>

- ◆ Trade-off between **time** and **space**<sup>6</sup>

- Scope handling<sup>7</sup>

- ◆ Stack: Inefficient searching and use of memory<sup>8</sup>

- ◆ So **chaining** is introduced<sup>9</sup>

- Static and dynamic typing and type checking<sup>10</sup>

- ◆ Static: **less bugs and execution time**, but **more effort**<sup>11</sup>

- ◆ Dynamic: **less effort**, but **more bugs and execution time**<sup>12</sup>

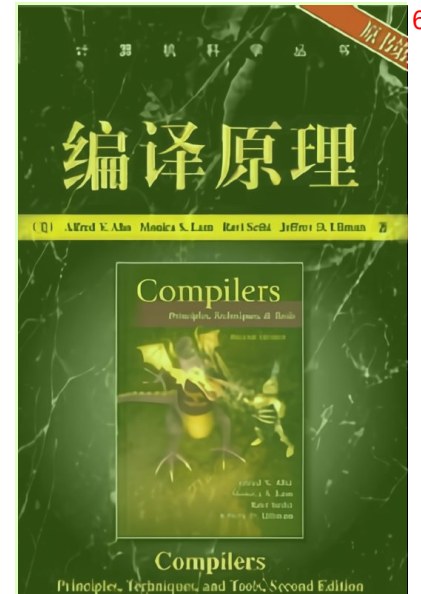
- ◆ Static ones are more desirable, but is often insufficient, e.g., downcasting<sup>13</sup>

# Further Reading<sup>1</sup>

## • Dragon Book, 2<sup>nd</sup> Edition<sup>2</sup>

### ◆ Comprehensive Reading:<sup>3</sup>

- Section 5.1-5.3 on introduction to syntax-directed translation.<sup>4</sup>
- Section 5.4-5.5 on the implementation of translation schemes in top-down and bottom-up LR parsing.<sup>5</sup>





# 编译原理<sup>1</sup>

## Compiler Principles

### Exercise

### Syntax and Semantic Analysis

---

赵帅<sup>2</sup>

计算机学院<sup>3</sup>

中山大学<sup>4</sup>



# Exercise<sup>1</sup>

- 对于一个无二义性文法G，它的一个句子的最左与最右推导产生的分析树一定是一样的吗?<sup>2</sup>
  - 对的，是这样的。语法树会隐去推导顺序<sup>3</sup>
- 怎么样判断一个文法是否为LL(1)型文法?<sup>4</sup>
  - 对文法中的每个产生式  $A \rightarrow \alpha \mid \beta$ :<sup>5</sup>
    - $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ .<sup>6</sup>
    - If  $\beta \Rightarrow \epsilon$ , then  $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$ .<sup>7</sup>
- LL(1)语法分析表行数与列数分别由哪些因素确定?<sup>8</sup>
  - 行数是该文法的非终结符个数，列数为终结符个数+1（结束符号\$）<sup>9</sup>
- 设句型aaAb的最右推导为  $S \Rightarrow aBb \Rightarrow aaAb$ ，句柄为aA，该句型的活前缀是?<sup>10</sup>
  - a, aa, aaA



# Exercise<sup>1</sup>

- 找出以下文法中的句型的所有短语，直接短语，句柄<sup>2</sup>

• 文法:  $G(E): E \rightarrow T \mid E+T; \quad T \rightarrow F \mid T*F; \quad F \rightarrow (E) \mid a \mid b \mid c$ <sup>3</sup>

• 句型:  $a * b + c$ <sup>4</sup>

- 第一步: 画出语法树<sup>5</sup>

- 第二步: 找出语法树的叶子节点<sup>6</sup>

•  $a, *, b, +, c$

- 第三步: 由叶子节点找出短语<sup>7</sup>

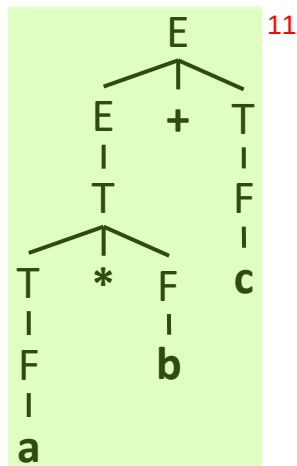
•  $a, a*b, b, c, a*b+c$  (why not  $*b, b+c, *, +?$ )<sup>8</sup>

- 第四步: 从短语中找出直接短语<sup>9</sup>

•  $a, b, c$

- 第五步: 从直接短语中找出句柄<sup>10</sup>

•  $a$



# Exercise<sup>1</sup>

- 给定一下文法 $G(S)$ , 请构建该文法的**LL(1)**分析表<sup>2</sup>

- $S \rightarrow A$ <sup>3</sup>
- $A \rightarrow \varepsilon$ <sup>4</sup>
- $A \rightarrow bbA$ <sup>5</sup>

- FIRST & Follow:**<sup>6</sup>

- $S \rightarrow A$ :  $\text{FIRST}(\alpha) = \{b, \varepsilon\}$ <sup>7</sup>
- $A \rightarrow \varepsilon$ :  $\text{FIRST}(\alpha) = \{\varepsilon\}$ <sup>8</sup>
- $A \rightarrow bbA$ :  $\text{FIRST}(\alpha) = \{b\}$ <sup>9</sup>
- $\text{FOLLOW}(S) = \{\$ \}$ <sup>10</sup>
- $\text{FOLLOW}(A) = \{\$ \}$ <sup>11</sup>

- Parse Table:**<sup>12</sup>

| b |                     | \$                          |            |
|---|---------------------|-----------------------------|------------|
| S |                     |                             |            |
| A |                     |                             |            |
| b |                     | \$                          | use FIRST  |
| S | $S \rightarrow A$   |                             |            |
| A | $A \rightarrow bbA$ |                             |            |
| b |                     | \$                          | use FOLLOW |
| S | $S \rightarrow A$   | $S \rightarrow A$           |            |
| A | $A \rightarrow bbA$ | $A \rightarrow \varepsilon$ |            |



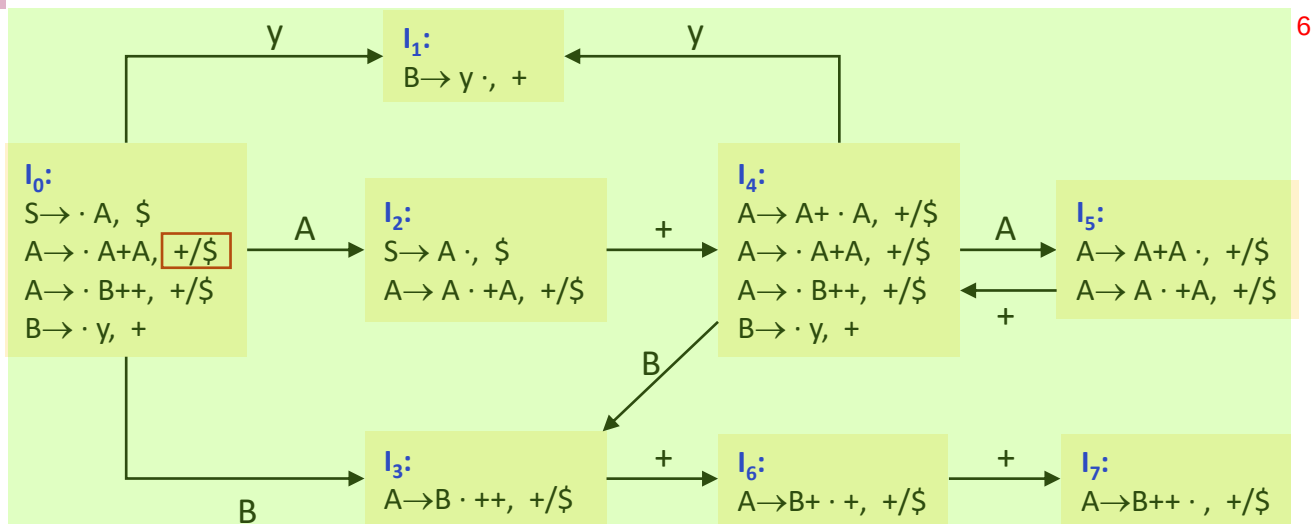
# Exercise<sup>1</sup>

• 给定一下文法G(S), 请构建识别活前缀的DFA与LR(1)分析表<sup>2</sup>

•  $S \rightarrow A$ <sup>3</sup>

•  $A \rightarrow A + A \mid B + +$ <sup>4</sup>

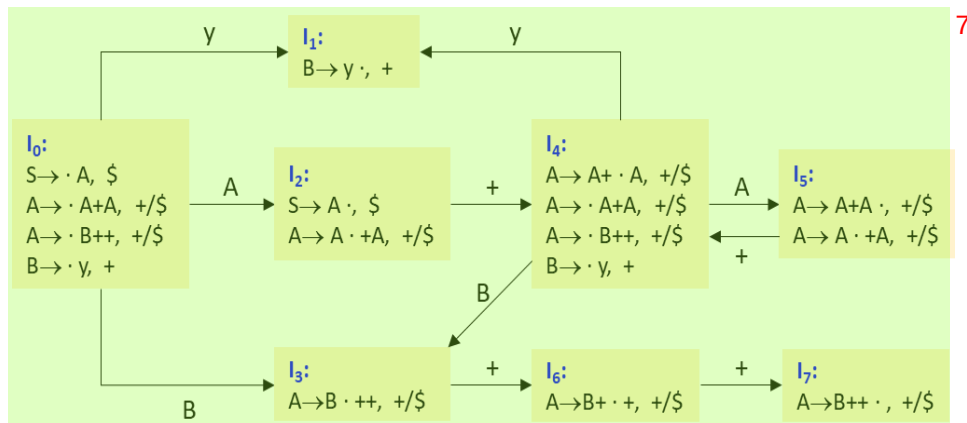
•  $B \rightarrow y$ <sup>5</sup>



# Exercise<sup>1</sup>

• 给定一下文法G(S), 请构建识别活前缀的DFA与LR(1)分析表<sup>2</sup>

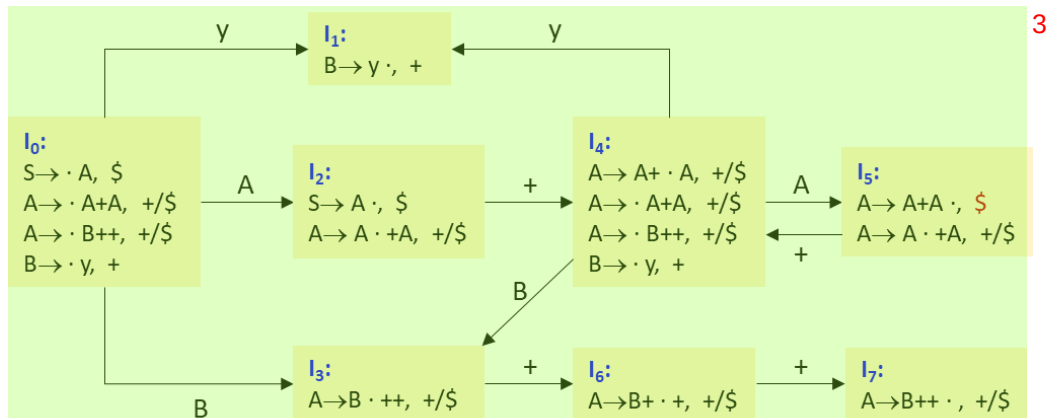
- (1)  $S \rightarrow A$ <sup>3</sup>
- (2)  $A \rightarrow A + A$ <sup>4</sup>
- (3)  $A \rightarrow B ++$ <sup>5</sup>
- (4)  $B \rightarrow y$ <sup>6</sup>



| S | ACTION |       |     | GOTO |   |
|---|--------|-------|-----|------|---|
|   | y      | +     | \$  | A    | B |
| 0 | s1     |       |     | 2    | 3 |
| 1 |        | r4    |     |      |   |
| 2 |        | s4    | acc |      |   |
| 3 |        | s6    |     |      |   |
| 4 | s1     |       |     | 5    | 3 |
| 5 |        | r2/s4 | r2  |      |   |
| 6 |        | s7    |     |      |   |
| 7 |        | r3    | r3  |      |   |

# Exercise<sup>1</sup>

- 该文法是LR(1)型文法吗? 若不是, 应如何解决? (有多种选择)<sup>2</sup>



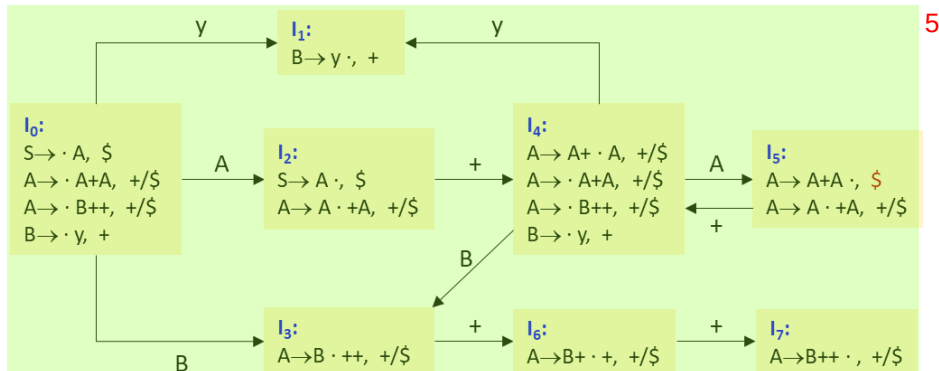
| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

# Exercise<sup>1</sup>

• 对于句子y++, 请给出其分析流程, 并说明y++是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack    | Input | ACTION | GOTO |
|----------|-------|--------|------|
| <u>0</u> | \$    | y++\$  |      |



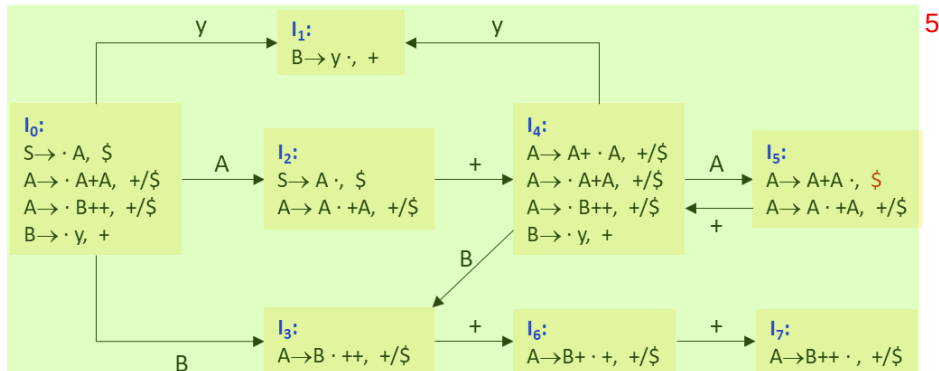
| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | r4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

# Exercise<sup>1</sup>

• 对于句子y++, 请给出其分析流程, 并说明y++是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack    | Input | ACTION | GOTO |
|----------|-------|--------|------|
| <u>0</u> | \$    | y++\$  | s1   |



| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

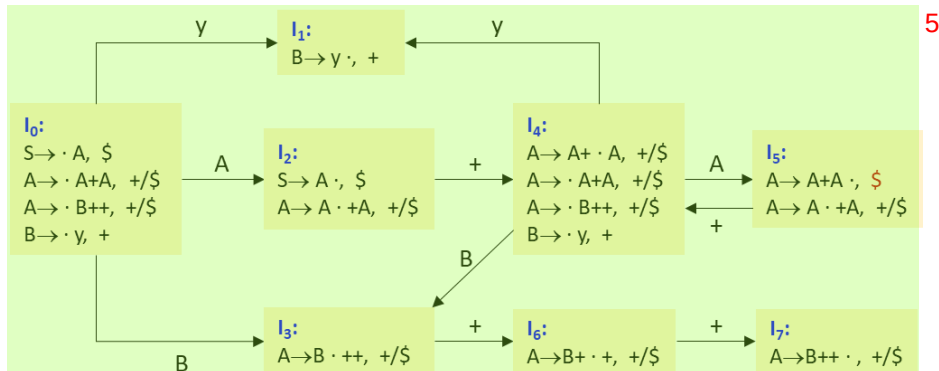


# Exercise<sup>1</sup>

• 对于句子y++, 请给出其分析流程, 并说明y++是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack      | Input | ACTION       | GOTO |
|------------|-------|--------------|------|
| <u>0</u>   | \$    | y++\$        | s1   |
| 0 <u>1</u> | \$y   | <u>+</u> +\$ |      |



6

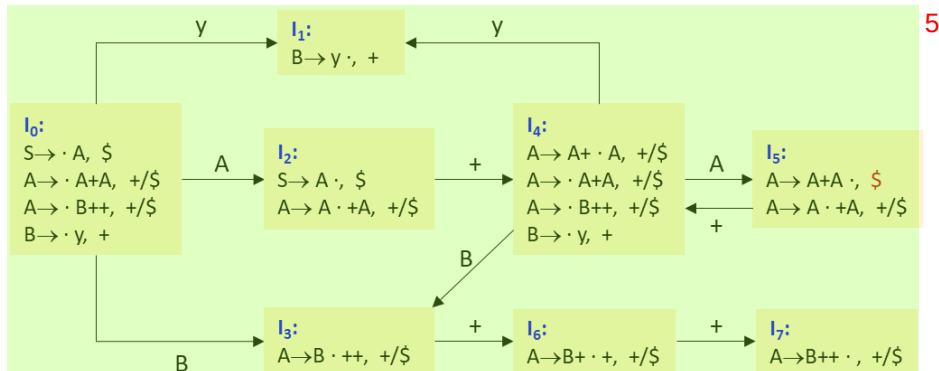
| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

# Exercise<sup>1</sup>

• 对于句子y++, 请给出其分析流程, 并说明y++是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack      | Input | ACTION        | GOTO                    |
|------------|-------|---------------|-------------------------|
| <u>0</u>   | \$    | <u>y</u> ++\$ | s1                      |
| 0 <u>1</u> | \$y   | <u>+</u> +\$  | r4( $B \rightarrow y$ ) |



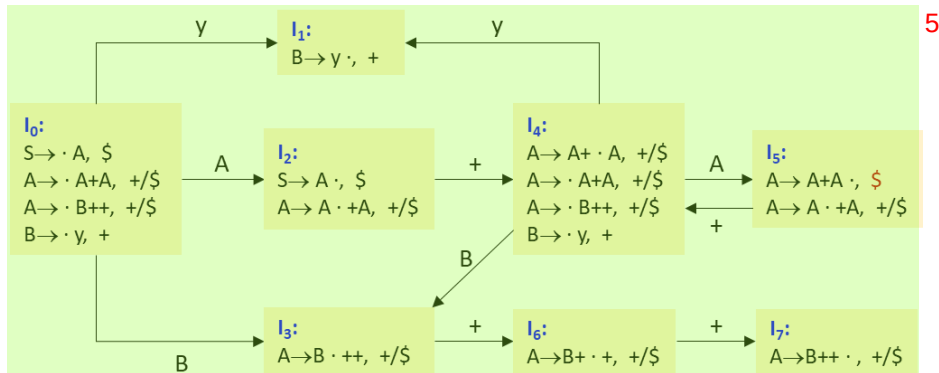
| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

# Exercise<sup>1</sup>

• 对于句子y++, 请给出其分析流程, 并说明y++是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack      | Input | ACTION | GOTO                    |
|------------|-------|--------|-------------------------|
| <u>0</u>   | \$    | y++\$  | s1                      |
| 0 <u>1</u> | \$y   | ++\$   | r4( $B \rightarrow y$ ) |
| 0          | \$B   | ++\$   | 3                       |



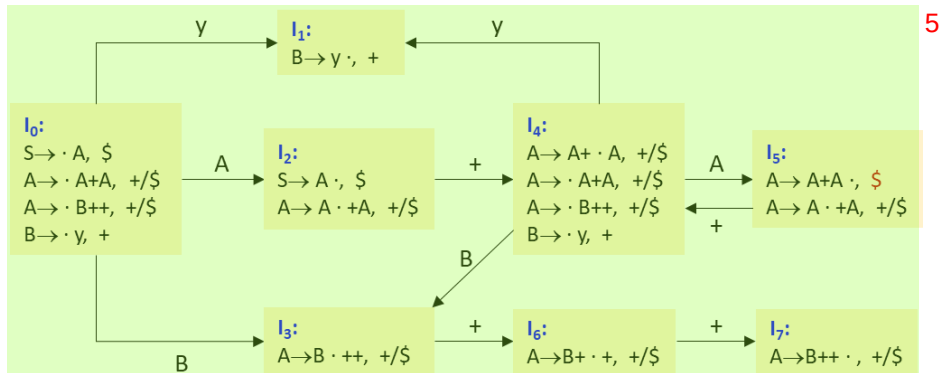
| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

# Exercise<sup>1</sup>

• 对于句子 $y++$ , 请给出其分析流程, 并说明 $y++$ 是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack      | Input | ACTION        | GOTO                    |
|------------|-------|---------------|-------------------------|
| <u>0</u>   | \$    | <u>y</u> ++\$ | s1                      |
| 0 <u>1</u> | \$y   | <u>++</u> \$  | r4( $B \rightarrow y$ ) |
| 0          | \$B   | <u>++</u> \$  | 3                       |
| 0 <u>3</u> | \$B   | <u>++</u> \$  | s6                      |



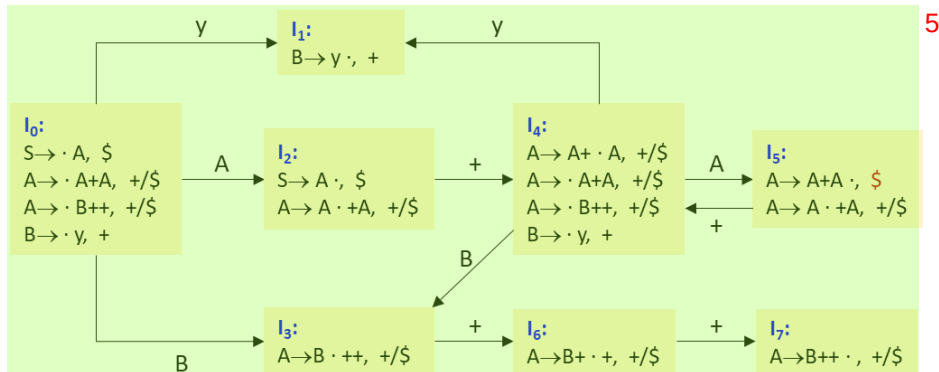
| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

# Exercise<sup>1</sup>

• 对于句子y++, 请给出其分析流程, 并说明y++是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack       | Input | ACTION | GOTO                    |
|-------------|-------|--------|-------------------------|
| <u>0</u>    | \$    | y++\$  | s1                      |
| 0 <u>1</u>  | \$y   | ++\$   | r4( $B \rightarrow y$ ) |
| 0           | \$B   | ++\$   | 3                       |
| 0 <u>3</u>  | \$B   | ++\$   | s6                      |
| 03 <u>6</u> | \$B+  | +\$    | s7                      |



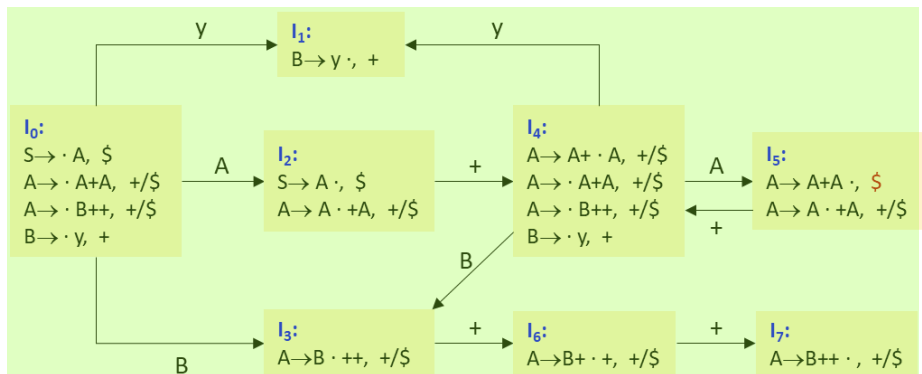
| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

# Exercise<sup>1</sup>

• 对于句子y++, 请给出其分析流程, 并说明y++是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack        | Input | ACTION        | GOTO                       |
|--------------|-------|---------------|----------------------------|
| <u>0</u>     | \$    | <u>y</u> ++\$ | s1                         |
| 0 <u>1</u>   | \$y   | <u>++</u> \$  | r4( $B \rightarrow y$ )    |
| 0            | \$B   | <u>++</u> \$  | 3                          |
| 0 <u>3</u>   | \$B   | <u>++</u> \$  | s6                         |
| 03 <u>6</u>  | \$B+  | <u>+</u> \$   | s7                         |
| 036 <u>7</u> | \$B++ | <u>\$</u>     | r3( $A \rightarrow B ++$ ) |



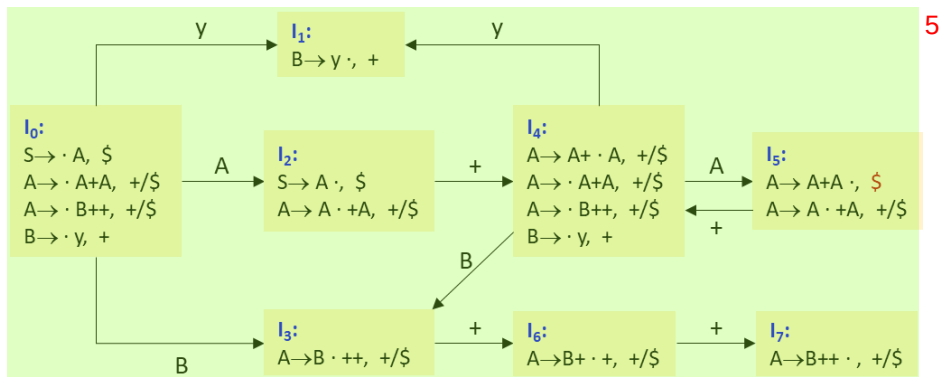
| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

# Exercise<sup>1</sup>

• 对于句子y++, 请给出其分析流程, 并说明y++是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack        | Input | ACTION | GOTO                       |
|--------------|-------|--------|----------------------------|
| <u>0</u>     | \$    | y++\$  | s1                         |
| 0 <u>1</u>   | \$y   | ++\$   | r4( $B \rightarrow y$ )    |
| 0            | \$B   | ++\$   | 3                          |
| 0 <u>3</u>   | \$B   | ++\$   | s6                         |
| 03 <u>6</u>  | \$B+  | +\$    | s7                         |
| 036 <u>7</u> | \$B++ | \$     | r3( $A \rightarrow B ++$ ) |
| 0            | \$A   | \$     | 2                          |



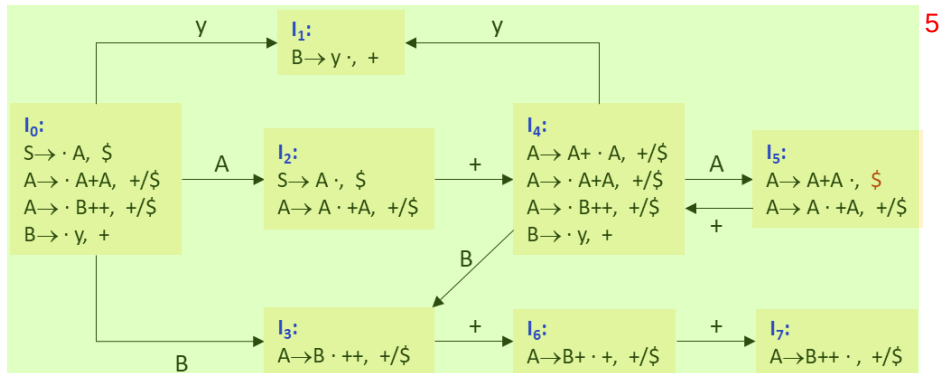
| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |

# Exercise<sup>1</sup>

• 对于句子y++, 请给出其分析流程, 并说明y++是否为该文法的一个句子?<sup>2</sup>

(1)  $S \rightarrow A$ ; (2)  $A \rightarrow A + A$ ; (3)  $A \rightarrow B ++$ ; (4)  $B \rightarrow y$ ;<sup>3</sup>

| Stack        | Input | ACTION | GOTO                       |
|--------------|-------|--------|----------------------------|
| <u>0</u>     | \$    | y++\$  | s1                         |
| 0 <u>1</u>   | \$y   | ++\$   | r4( $B \rightarrow y$ )    |
| 0            | \$B   | ++\$   | 3                          |
| 0 <u>3</u>   | \$B   | ++\$   | s6                         |
| 03 <u>6</u>  | \$B+  | +\$    | s7                         |
| 036 <u>7</u> | \$B++ | \$     | r3( $A \rightarrow B ++$ ) |
| 0            | \$A   | \$     | 2                          |
| 0 <u>2</u>   | \$A   | \$     | acc                        |



| S | ACTION |    |     | GOTO |   |
|---|--------|----|-----|------|---|
|   | y      | +  | \$  | A    | B |
| 0 | s1     |    |     | 2    | 3 |
| 1 |        | r4 |     |      |   |
| 2 |        | s4 | acc |      |   |
| 3 |        | s6 |     |      |   |
| 4 | s1     |    |     | 5    | 3 |
| 5 |        | s4 | r2  |      |   |
| 6 |        | s7 |     |      |   |
| 7 |        | r3 | r3  |      |   |



# Exercise<sup>1</sup>

给定文法G(S):<sup>2</sup>

$$S \rightarrow Sb \mid bAa$$
$$A \rightarrow aSc \mid aSb \mid a$$

(1) 拓广文法后, 用LR(0)项目构造一个识别G(S)所有活前缀的DFA<sup>3</sup>

(2) 判断该文法是否为LR(0)文法, 并阐明原因<sup>4</sup>

(3) 判断该文法是否为LR(1)文法, 并阐明原因。<sup>5</sup>

(4) 试构造该文法的SLR(1)分析表。<sup>6</sup>

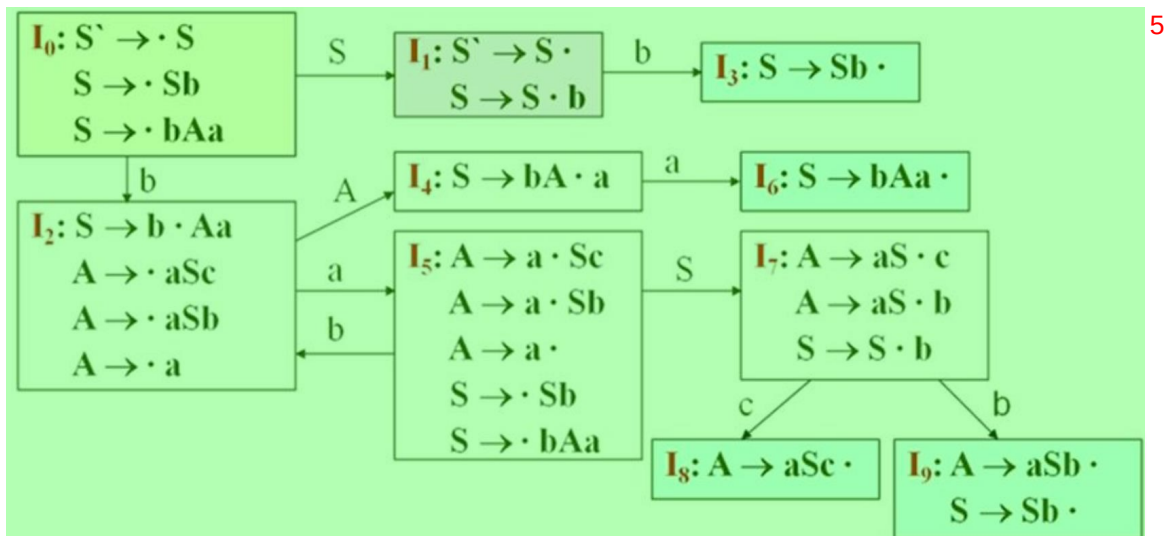
# Exercise<sup>1</sup>

给定文法G(S):<sup>2</sup>

$S \rightarrow Sb \mid bAa$ <sup>3</sup>

$A \rightarrow aSc \mid aSb \mid a$

(1) 拓广文法后, 用LR(0)项目构造一个识别G(S)所有活前缀的DFA<sup>4</sup>



# Exercise<sup>1</sup>

给定文法G(S):<sup>2</sup>

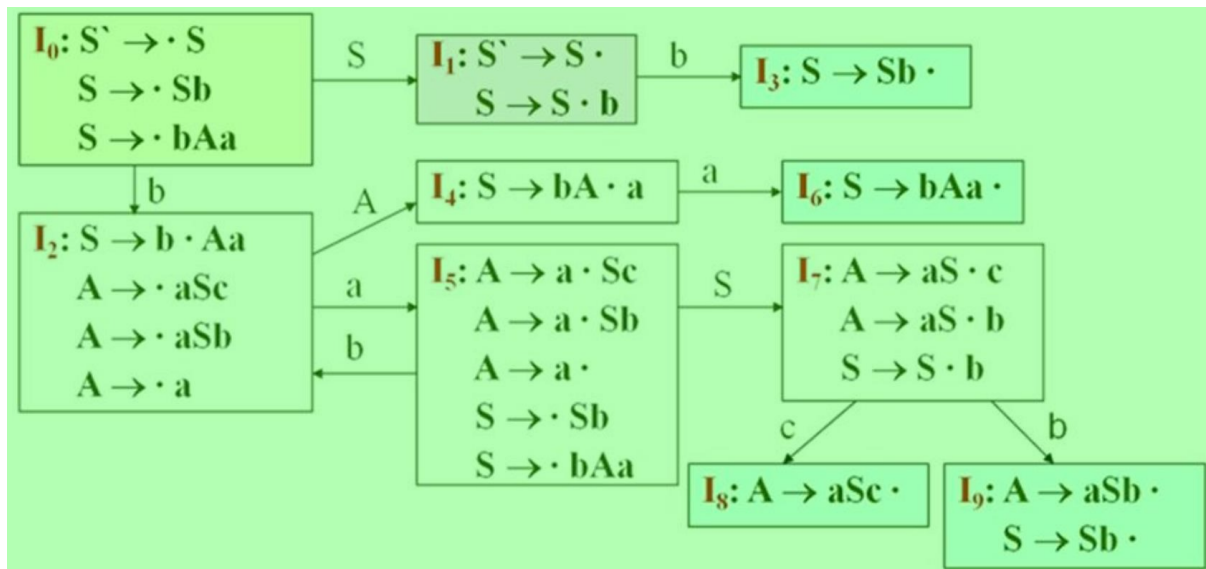
$S \rightarrow Sb \mid bAa$ <sup>3</sup>

$A \rightarrow aSc \mid aSb \mid a$ <sup>4</sup>

(2) 判断该文法是否为LR(0)文法，并阐明原因<sup>5</sup>

• 移进—归约:  $I_1, I_5$ <sup>6</sup>

• 归约—归约:  $I_9$ <sup>7</sup>



# Exercise<sup>1</sup>

给定文法G(S):<sup>2</sup>

$S \rightarrow Sb \mid bAa$ <sup>3</sup>

$A \rightarrow aSc \mid aSb \mid a$ <sup>4</sup>

(3) 判断该文法是否为LR(1)文法，并阐明原因<sup>5</sup>

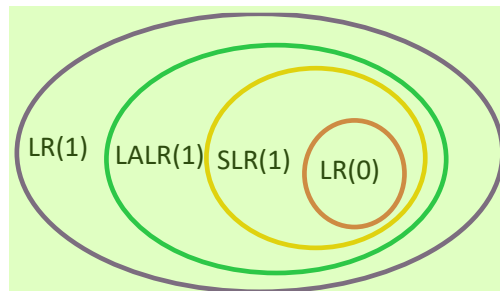
• 该文法是SLR(1)的，所以自然就是LR(1)文法。<sup>6</sup>

• 理由:<sup>7</sup>

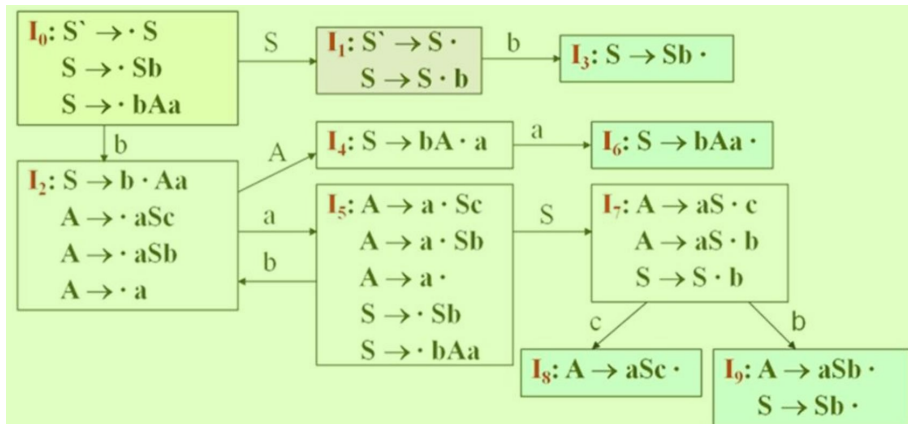
• I1: 由于 $b \notin \text{FOLLOW}(S')$ ，冲突可通过Lookahead解析<sup>8</sup>

• I5: 考虑 $b \notin \text{FOLLOW}(A)$ ，同理可解析<sup>9</sup>

• I9:  $\text{FOLLOW}(A) \cap \text{FOLLOW}(S) = \emptyset$ <sup>10</sup>



11



12

# Exercise<sup>1</sup>

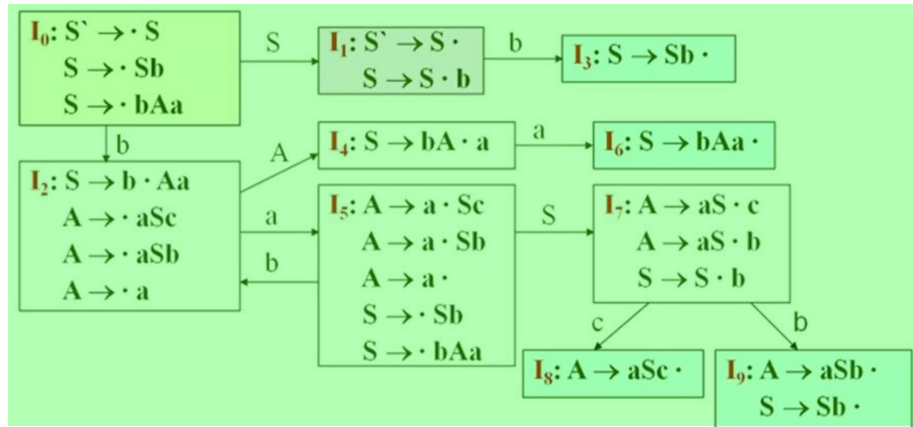


给定文法G(S):<sup>2</sup>

$S \rightarrow Sb \mid bAa$ <sup>3</sup>

$A \rightarrow aSc \mid aSb \mid a$ <sup>4</sup>

(4) 试构造该文法的SLR(1)分析表<sup>5</sup>



(1)  $S \rightarrow Sb$ <sup>6</sup>

(2)  $S \rightarrow bAa$ <sup>7</sup>

(3)  $A \rightarrow aSc$ <sup>8</sup>

(4)  $A \rightarrow aSb$ <sup>9</sup>

(5)  $A \rightarrow a$ <sup>10</sup>

推荐写增广后的文法<sup>11</sup>

|   | a    | b    | c    | #    | S  | A  |
|---|------|------|------|------|----|----|
| 0 |      | b2   |      |      | S1 |    |
| 1 |      | b3   |      | OK   |    |    |
| 2 | a5   |      |      |      |    | A4 |
| 3 | r(1) | r(1) | r(1) | r(1) |    |    |
| 4 | a6   |      |      |      |    |    |
| 5 | r(5) | b2   |      |      | S7 |    |
| 6 | r(2) | r(2) | r(2) | r(2) |    |    |
| 7 |      | b9   | c8   |      |    |    |
| 8 | r(3) | r(3) | r(3) | r(3) |    |    |
| 9 | r(4) | r(3) | r(3) | r(3) |    |    |

也可以按照课件中的分析表写法<sup>14</sup>

# Exercise<sup>1</sup>

## • 判断题<sup>2</sup>

- 在L-属性定义（L-Attributed Definition）中，每一个属性都是继承属性（Inherited Attribute）。<sup>3</sup>
  - 错。在L-属性定义中，要么是综合属性，要么是受限的继承属性，并非都是继承属性。<sup>4</sup>
- 如果一个翻译模式（Translation Scheme）中存在嵌入在产生式右部的左边或中间的语义动作，则该翻译模式只适合使用递归下降预测翻译器（Recursive Descent Predictive Translator）来实现，无法使用LR分析技术来实现。<sup>5</sup>
  - 错。翻译器分析技术的使用并不是由语义动作的实现决定，而是由文法的特点、语义动作的需求以及性能考虑决定。<sup>6</sup>

# Exercise<sup>1</sup>

已知文法  $G: S' \rightarrow S$ <sup>2</sup>

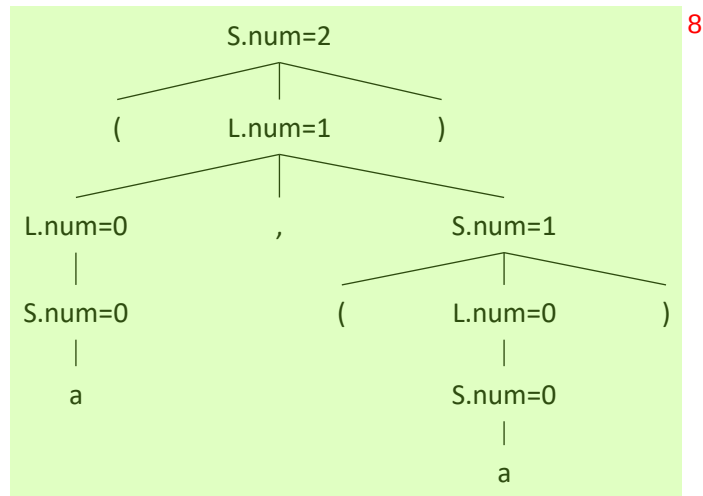
•  $S \rightarrow (L) \mid a$ <sup>3</sup>

•  $L \rightarrow L, S \mid S$ <sup>4</sup>

其语法制导翻译定义如下:<sup>5</sup>

若输入为  $(a,(a))$ ，且采用自底向上的分析方法，则输出为 2。<sup>6</sup>

| 产生式                    | 语义动作                      |
|------------------------|---------------------------|
| $S' \rightarrow S$     | print(S.num)              |
| $S \rightarrow (L)$    | $S.num = L.num + 1$       |
| $S \rightarrow a$      | $S.num = 0$               |
| $L \rightarrow L_1, S$ | $L.num = L_1.num + S.num$ |
| $L \rightarrow S$      | $L.num = S.num$           |

<sup>7</sup>

# Exercise<sup>1</sup>

假设我们有一个产生式 $A \rightarrow BCD$ 。A、B、C、D这四个非终结符号都有两个属性：<sup>2</sup>  
s是一个综合属性，而i是一个继承属性。对于下面的每组规则，请回答：这些规则是否满足S属性定义的要求？这些规则是否满足L属性定义的要求？并给出理由。

- $A.s = B.s + D.s$ <sup>3</sup>

- 该规则满足S属性定义的要求（每个属性都是综合属性），也满足L属性定义的要求（无继承属性）。<sup>4</sup>

- $A.s = B.i + C.s$ <sup>5</sup>

- 该规则不满足S属性定义的要求（存在继承属性i），满足L属性定义的要求（存在满足限制条件的继承属性i）。<sup>6</sup>

- $A.s = D.i, B.i = A.s + C.s, C.i = B.s, D.i = B.i + C.i$ <sup>7</sup>

- 该规则不满足S属性定义的要求（存在继承属性i），也不满足L属性定义的要求（存在不满足限制条件的继承属性B.i, 其依赖右兄弟的综合属性）。<sup>8</sup>



# Exercise<sup>1</sup>

考虑一个非标准的二进制数值系统，其中的每个二进制数 $b$ 的值（value）被定义为从右至左<sup>2</sup>的非零二进制数字表示的十进制数的交替和。例如， $\text{value}(\epsilon) = 0$ ， $\text{value}(10) = 2^1 = 2$ ， $\text{value}(100100) = 2^2 - 2^5 = -28$ ， $\text{value}(11110) = 2^1 - 2^2 + 2^3 - 2^4 = -10$ 。已知表示非标准二进制数字 $b$ 的文法 $G$ 如下：

•  $S \rightarrow T \mid \epsilon$ <sup>3</sup>

•  $T \rightarrow 0T \mid 1T$ <sup>4</sup>

•  $T \rightarrow 0 \mid 1$ <sup>5</sup>

请在表中添加相应的语义动作以计算 $\text{value}(b)$ ，<sup>6</sup>

你只能使用 $\text{val}$ 和 $\text{tmp}$ 两个属性。

| 产生式                      | 语义动作 <sup>7</sup>                                                                           |
|--------------------------|---------------------------------------------------------------------------------------------|
| $S \rightarrow T$        | $S.\text{val} = T.\text{val};$                                                              |
| $S \rightarrow \epsilon$ | $S.\text{val} = 0;$                                                                         |
| $T_1 \rightarrow 0T_2$   | $T_1.\text{tmp} = T_2.\text{tmp} * 2;$<br>$T_1.\text{val} = T_2.\text{val};$                |
| $T_1 \rightarrow 1T_2$   | $T1.\text{tmp} = T2.\text{tmp} * (-2);$<br>$T1.\text{val} = T2.\text{val} + T1.\text{tmp};$ |
| $T \rightarrow 0$        | $T.\text{tmp} = -1;$<br>$T.\text{val} = 0;$                                                 |
| $T \rightarrow 1$        | $T.\text{tmp} = 1;$<br>$T.\text{val} = 1;$ <sup>0</sup>                                     |